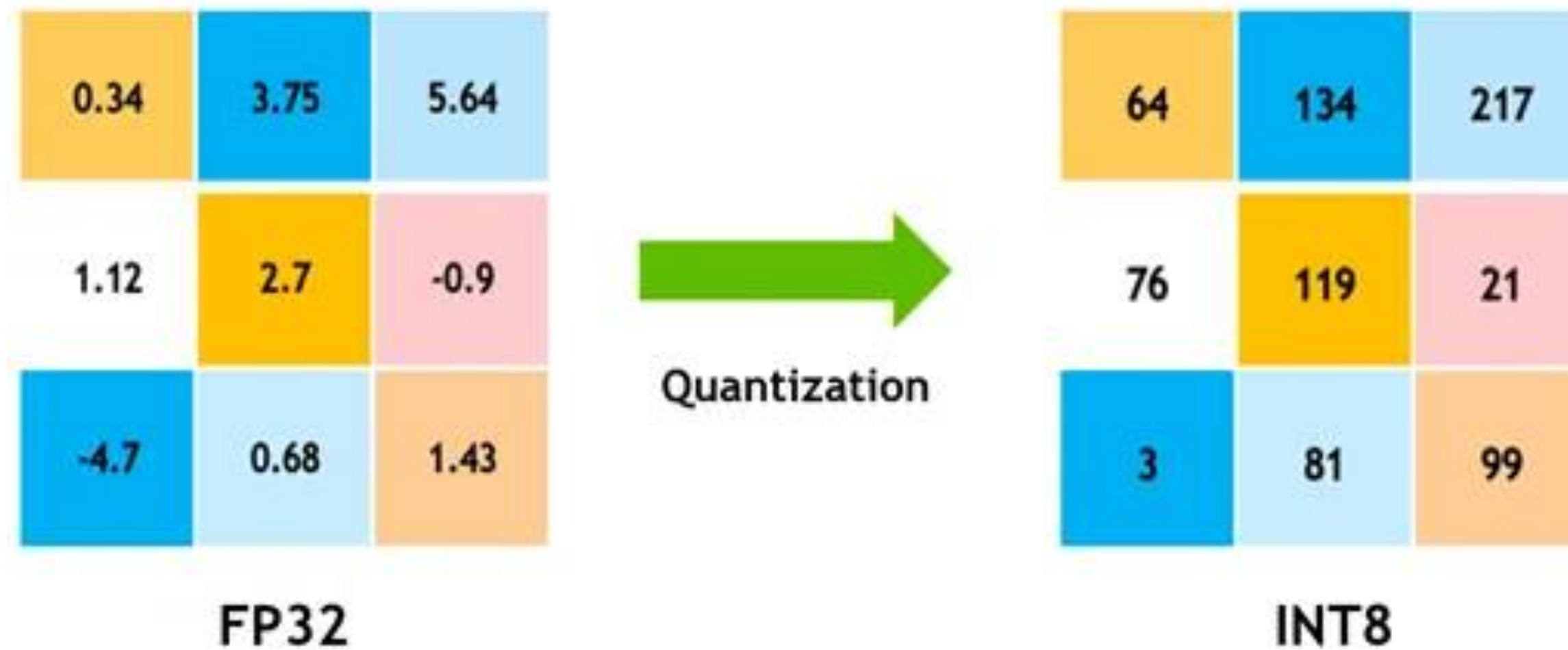
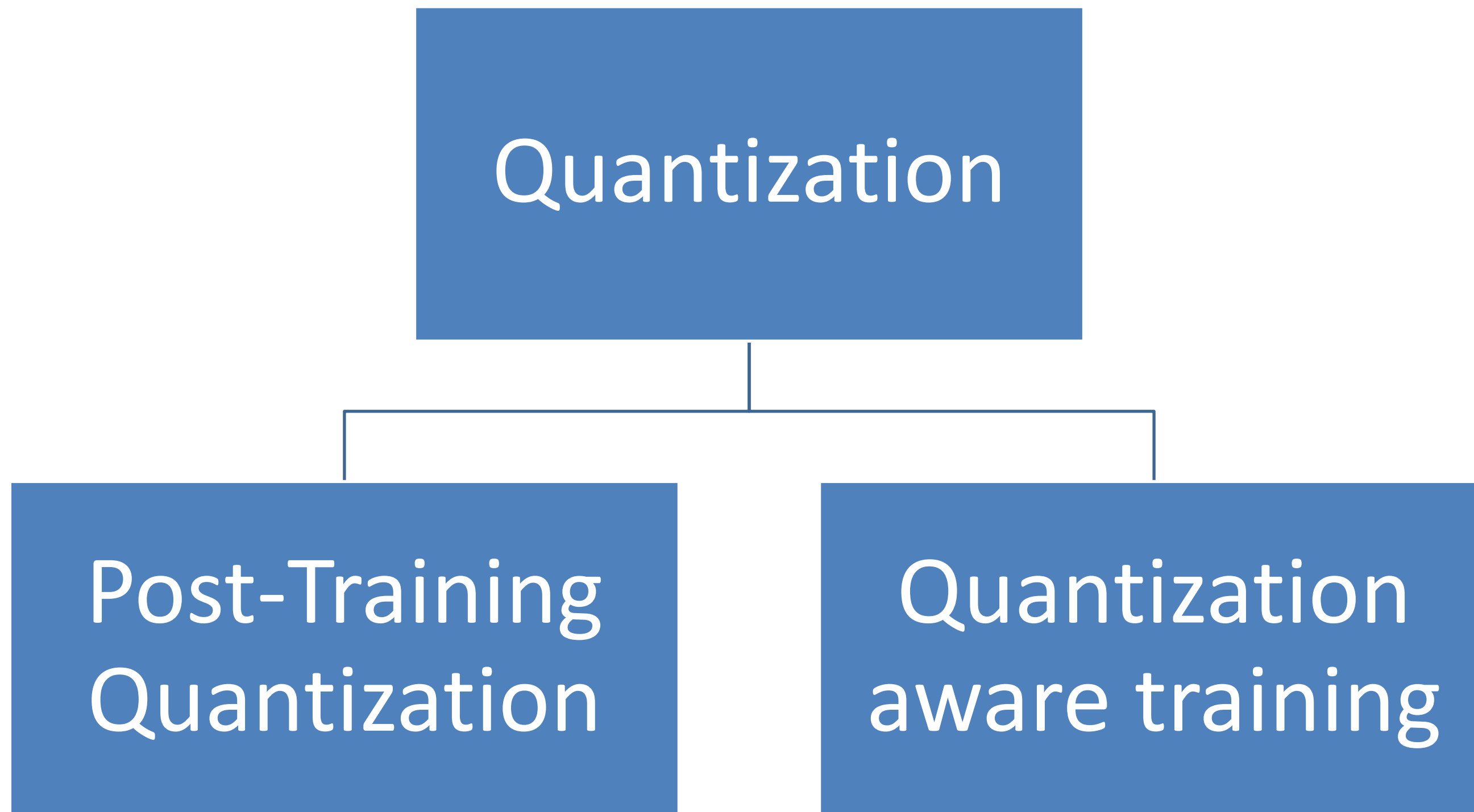


Qunatization

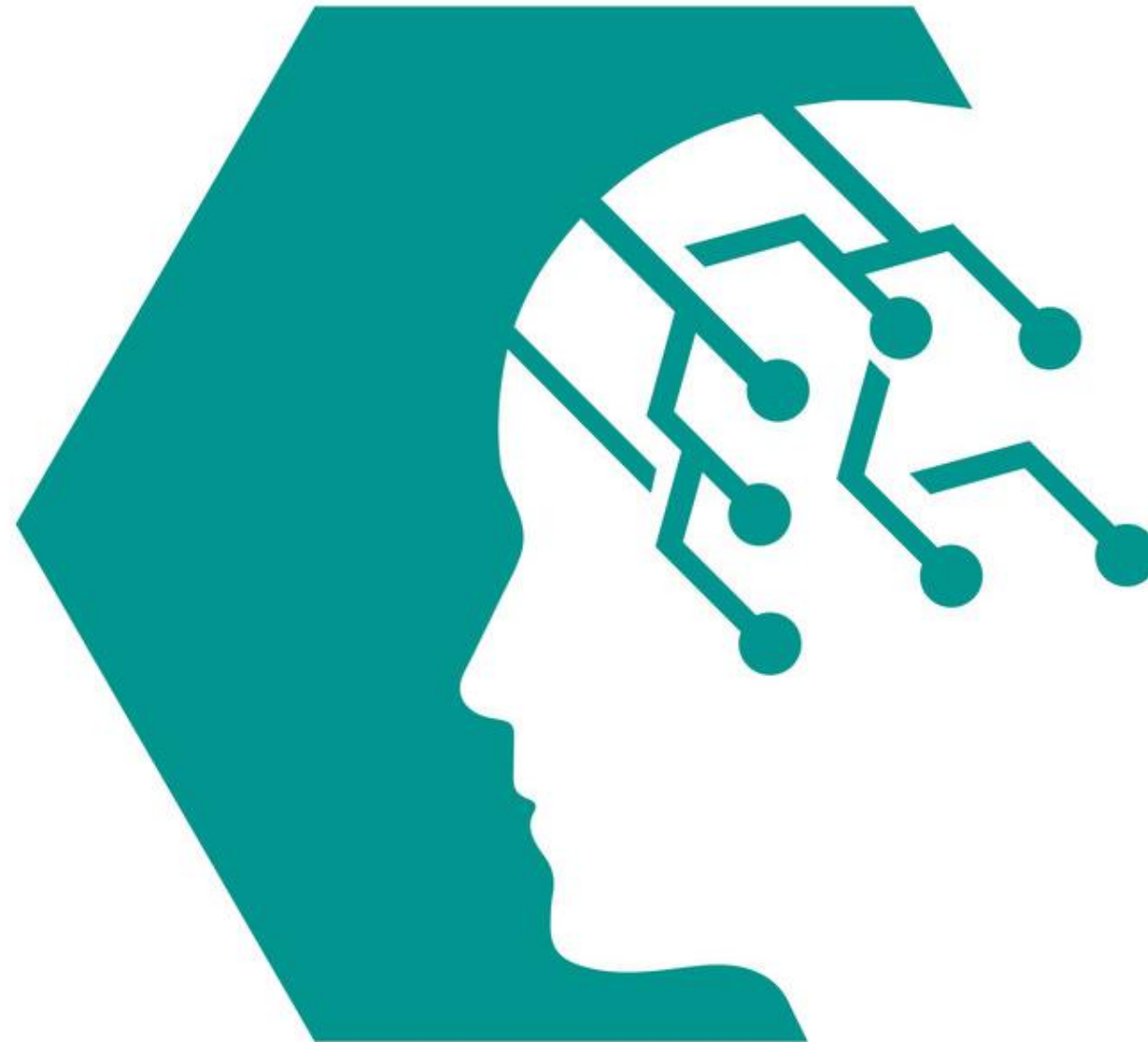
Quantization is a technique that reduces the precision of model parameters, thereby decreasing the number of bits needed to store each parameter.



Types of Quantization



Fine-tuning LLMs



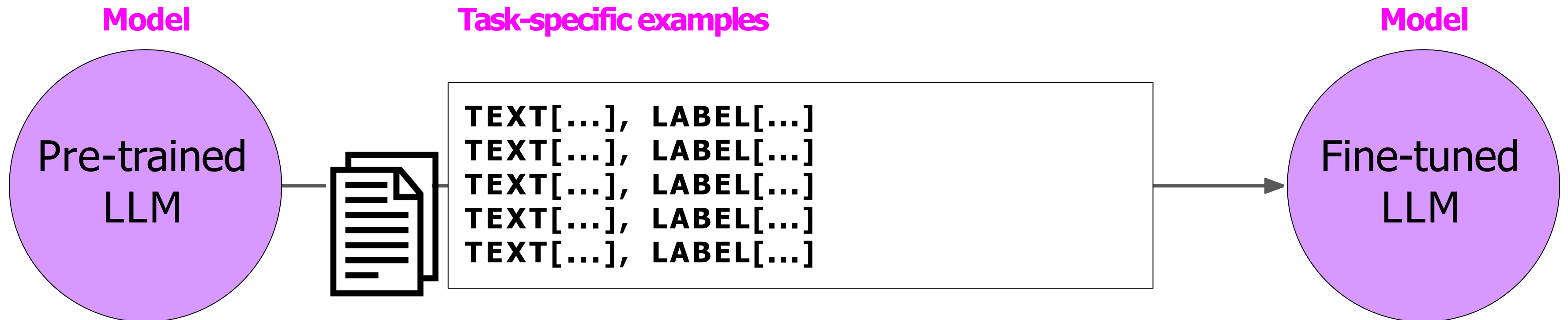
Need for Fine-tuning

- Fine-tuning LLMs (Large Language Models) is necessary to adapt pre-trained models for specific tasks or domains, improving their accuracy and relevance in specialized applications.
- While pre-trained LLMs possess broad knowledge, fine-tuning allows them to become experts in particular areas by learning domain-specific nuances and task-related patterns.



LLM fine-tuning at a high level

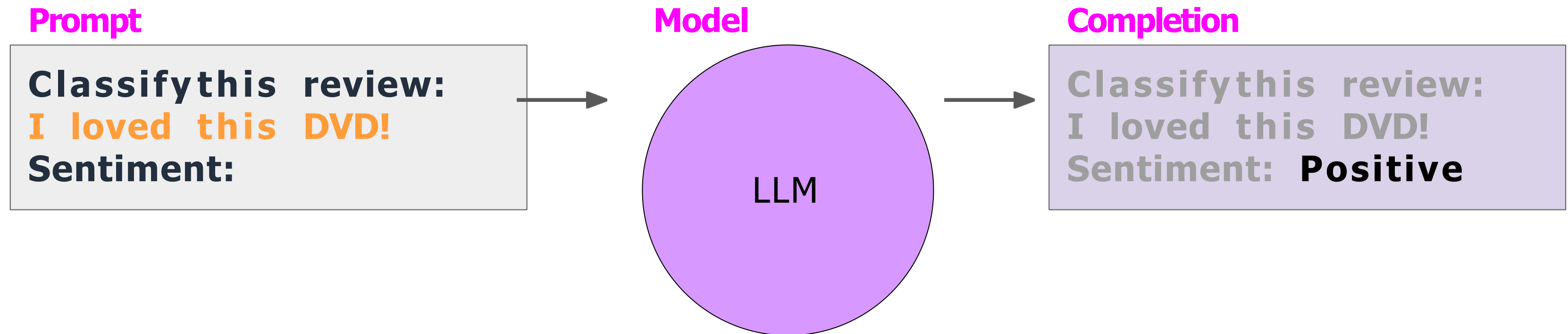
LLM fine-tuning



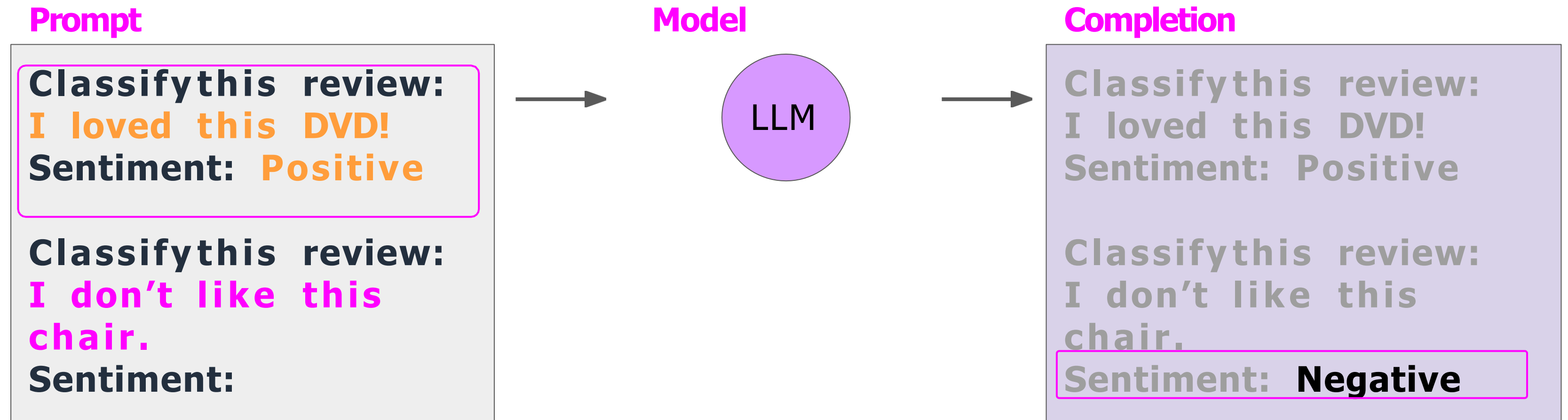
GB - TB
of labeled examples for a specific
task or set of tasks



In-context learning (ICL) - zero shot inference



In-context learning (ICL) - one/few shot inference



One-shot or Few-shot Inference



Need for Fine-tuning

Classify this review:

I loved this movie!

Sentiment: Positive

Classify this review:

I don't like this chair.

Sentiment: Negative

Classify this review:

This sofa is so ugly.

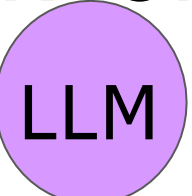
Sentiment: Negative

Classify this review:

Who would use this product?

Sentiment:

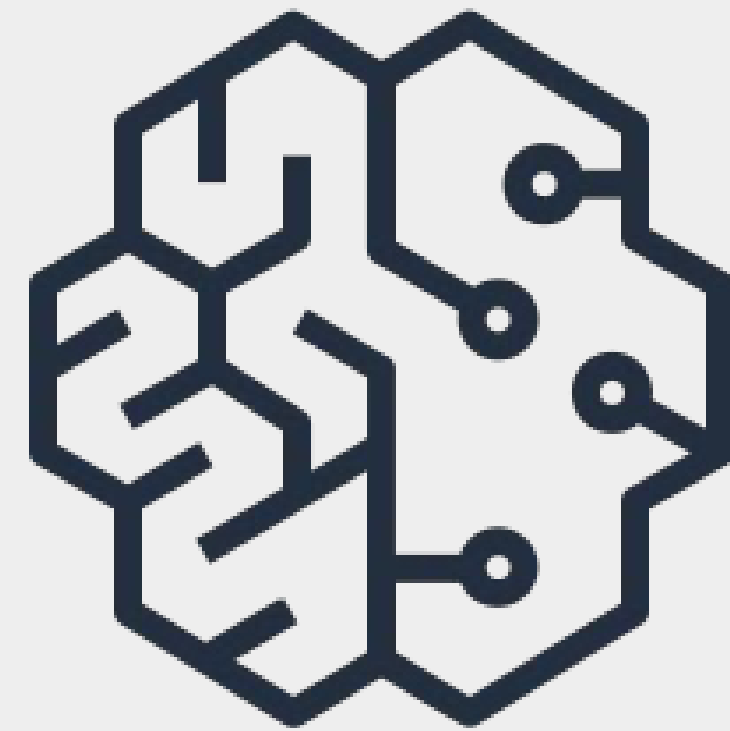
Even with
multiple
examples

- In-context learning may not work for smaller models 
- Examples take up space in the context window

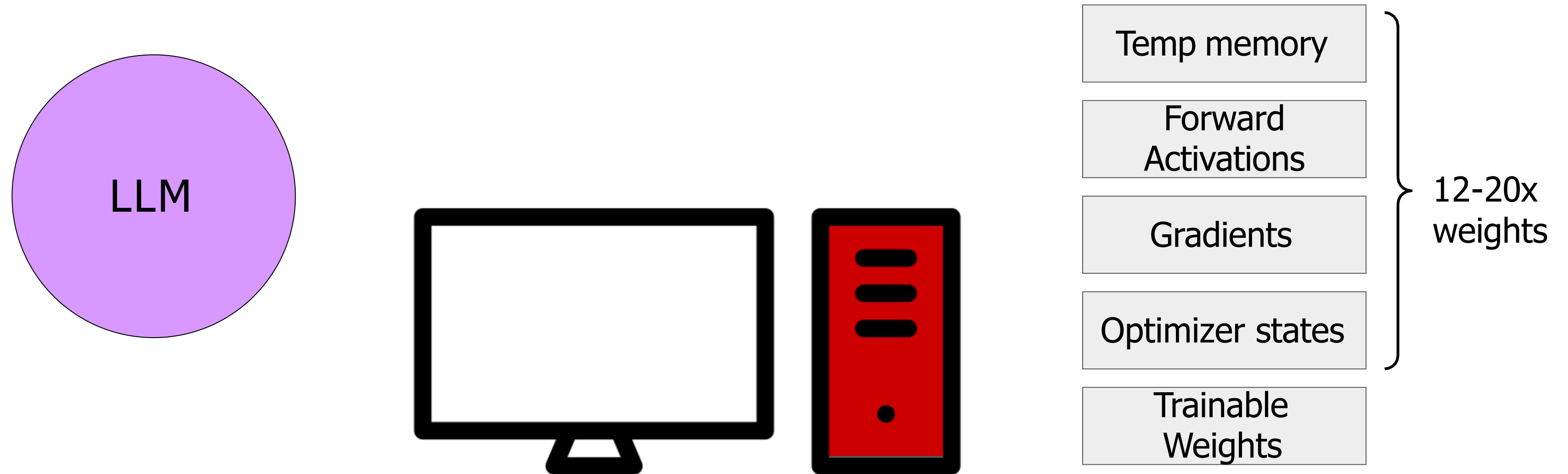
Instead, try **fine-tuning** the model



Parameter- efficient Fine-tuning (PEFT)



Full fine-tuning of large LLMs is challenging



Catastrophic Forgetting



Catastrophic Forgetting

Email SPAM classification

1



Fine-tuning on new data
(social media posts)

2

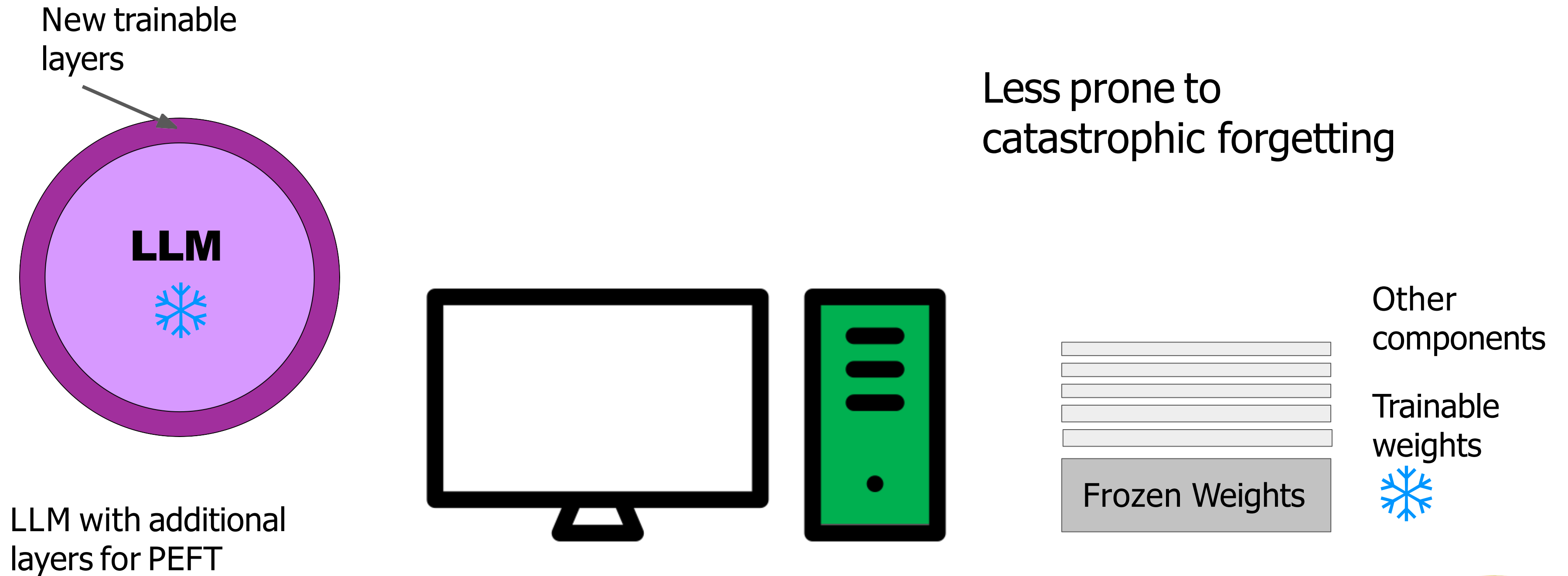


Model forgets previously-learned information

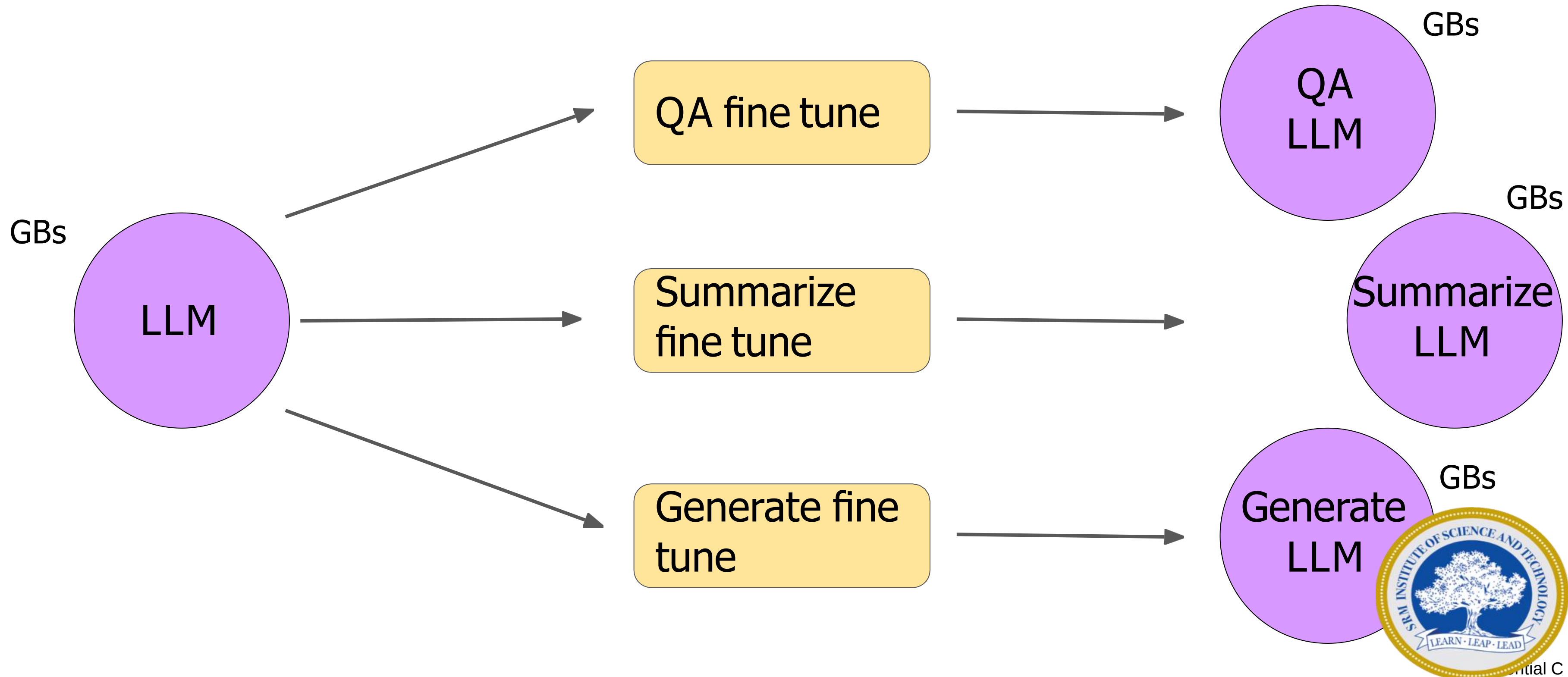
3



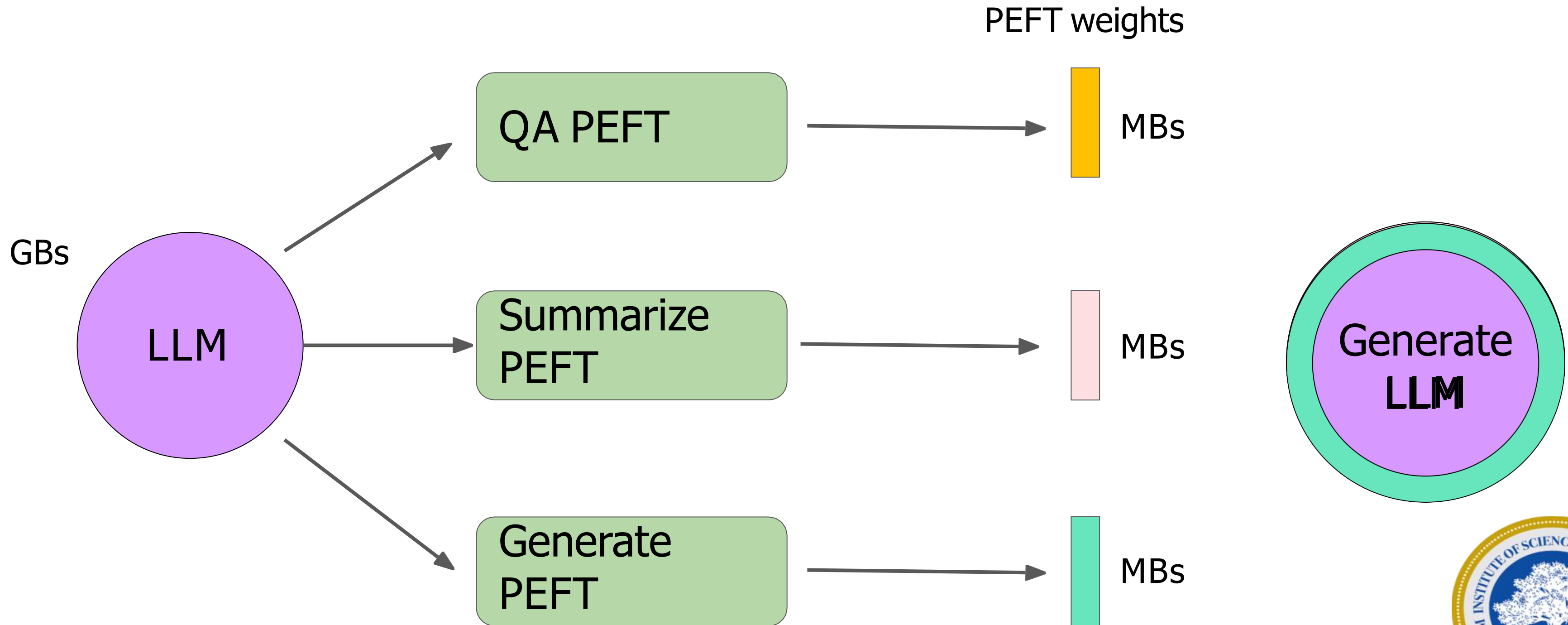
Parameter efficient fine-tuning (PEFT)



Full fine-tuning creates full copy of original LLM per task



PEFT fine-tuning saves space and is flexible



PEFT Methods

Selective

- *Subset of Initial Params to fine-tune*

Re-Parameterization

- *LoRA*
- *qLORA*

Additive

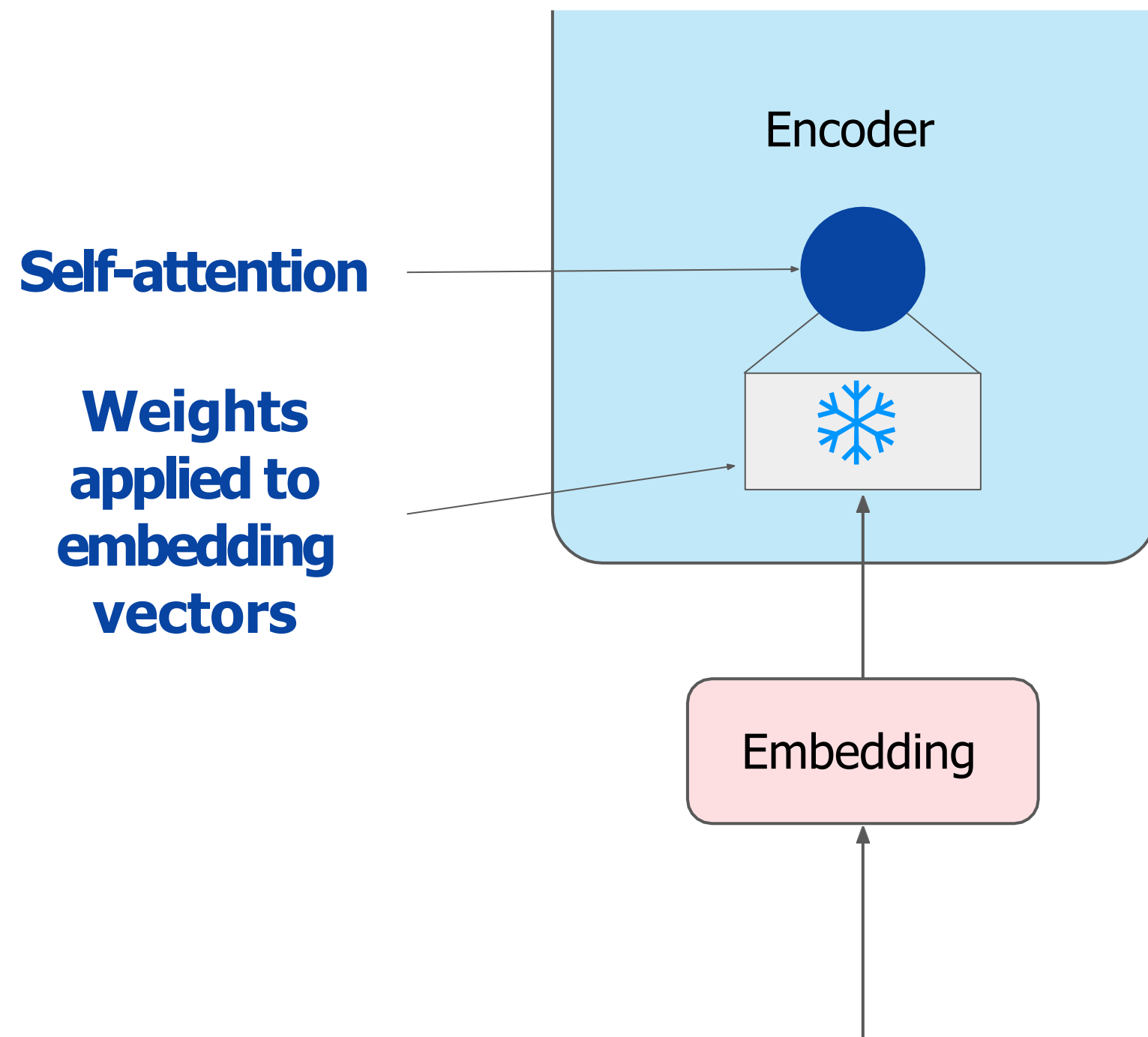
- *Adapters*
- *Soft Prompting*



Low-Rank Adaptation of Large Language Models



LoRA: Low Rank Adaption of LLMs



1. Freeze most of the original LLM weights.



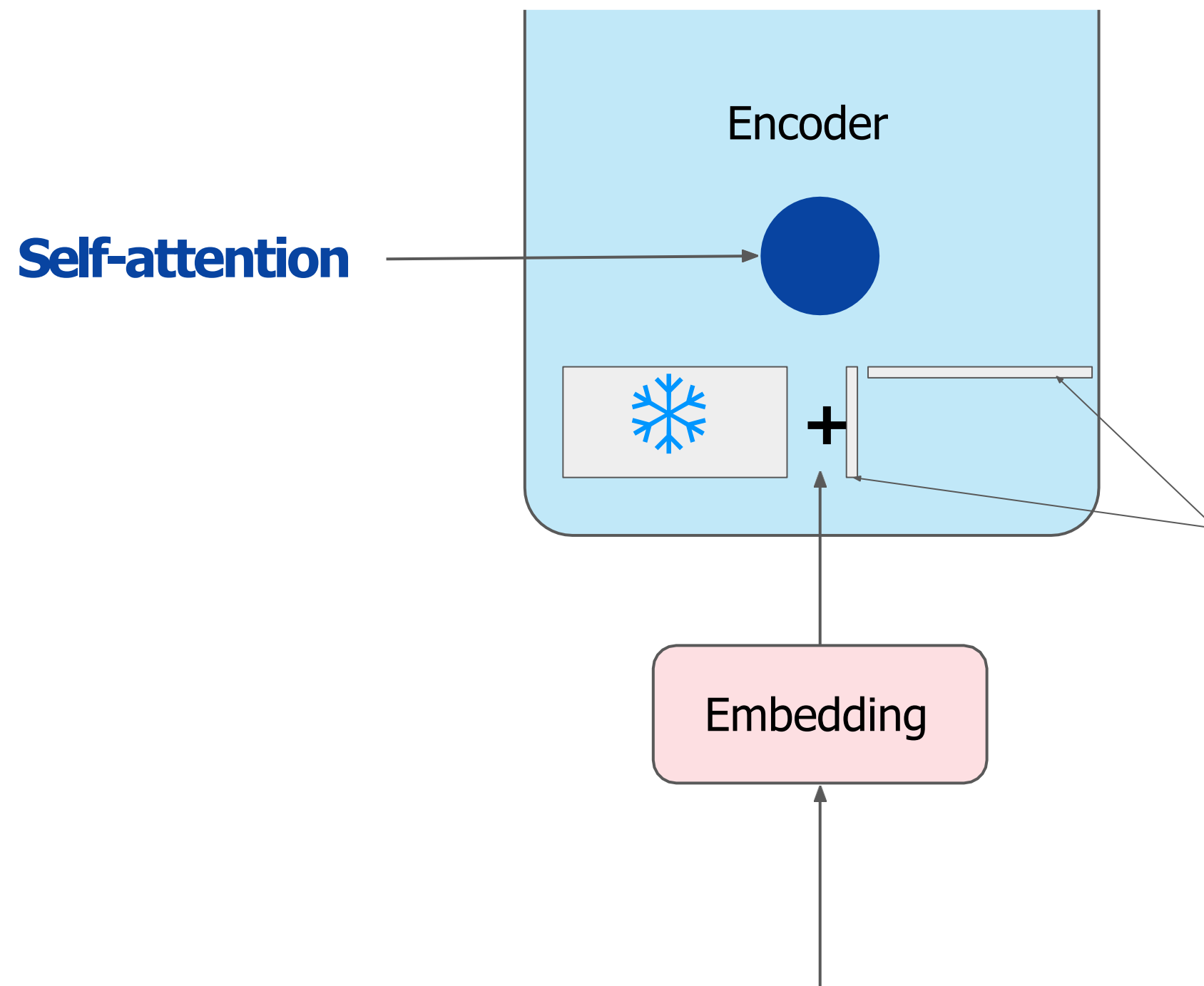
LoRA: Low Rank Adaption of LLMs

1. Freeze most of the original LLM weights.

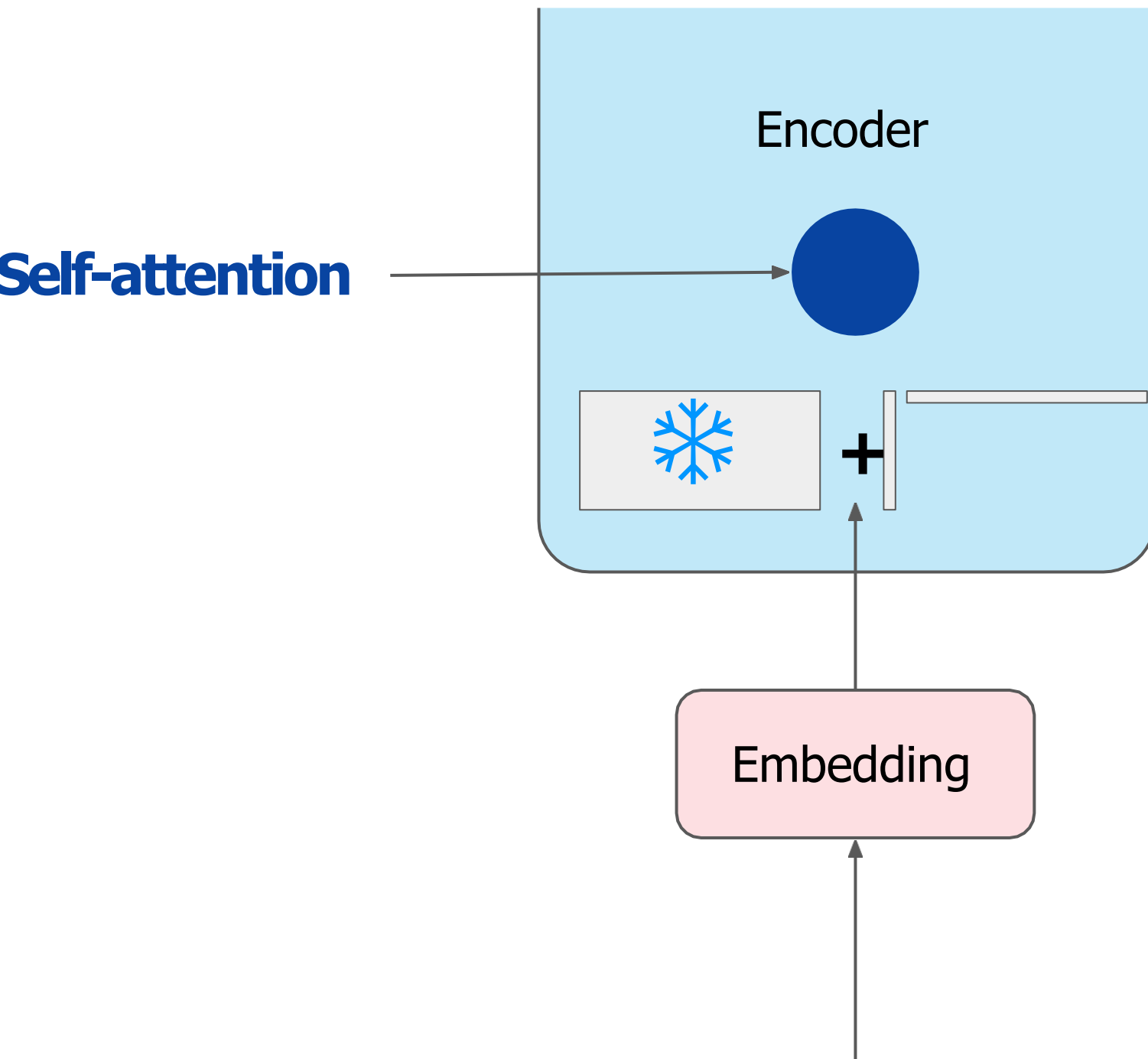
2. Inject 2 **rank decomposition matrices**

3. Train the weights of the smaller matrices

Rank r is small dimension, typically 4, 8 ... 64



LoRA: Low Rank Adaption of LLMs



1. Freeze most of the original LLM weights.
2. Inject 2 **rank decomposition matrices**
3. Train the weights of the smaller matrices

Steps to update model for inference

1. Matrix multiply the low rank matrices

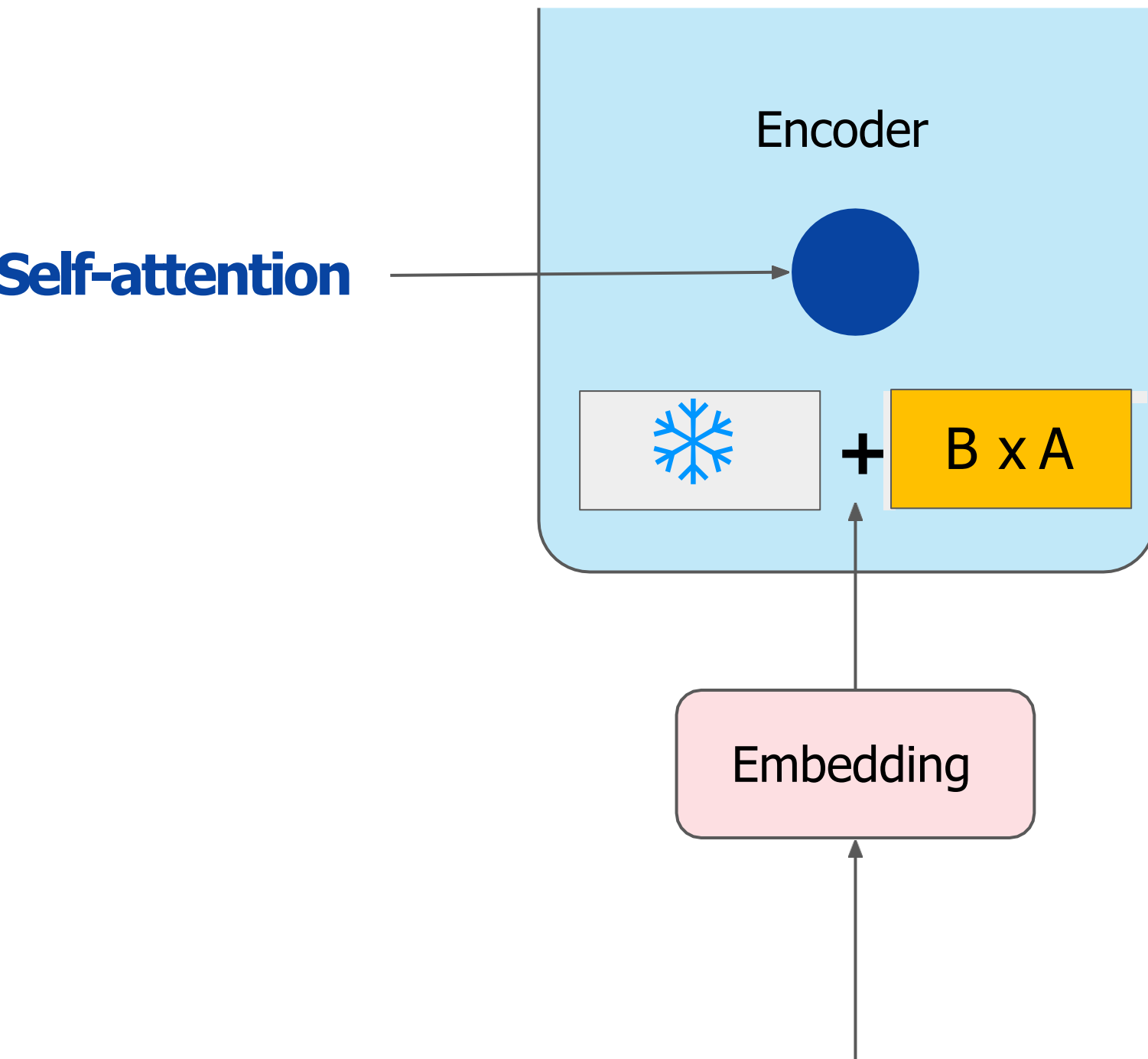
$$\begin{array}{|c|} \hline B \\ \hline \end{array} * \begin{array}{|c|} \hline A \\ \hline \end{array} = \begin{array}{|c|} \hline B \times A \\ \hline \end{array}$$

2. Add to original weights

$$\begin{array}{|c|} \hline \text{Snowflake} \\ \hline \end{array} + \begin{array}{|c|} \hline B \times A \\ \hline \end{array}$$



LoRA: Low Rank Adaption of LLMs



1. Freeze most of the original LLM weights.
2. Inject 2 **rank decomposition matrices**
3. Train the weights of the smaller matrices

Steps to update model for inference

1. Matrix multiply the low rank matrices

$$\begin{array}{|c|} \hline B \\ \hline \end{array} * \begin{array}{|c|} \hline A \\ \hline \end{array} = \begin{array}{|c|} \hline B \times A \\ \hline \end{array}$$

2. Add to original weights

$$\begin{array}{|c|} \hline \text{Snowflake} \\ \hline \end{array} + \begin{array}{|c|} \hline B \times A \\ \hline \end{array}$$



Concrete example using base Transformer as reference

Use the base Transformer model presented by Vaswani et al. 2017:

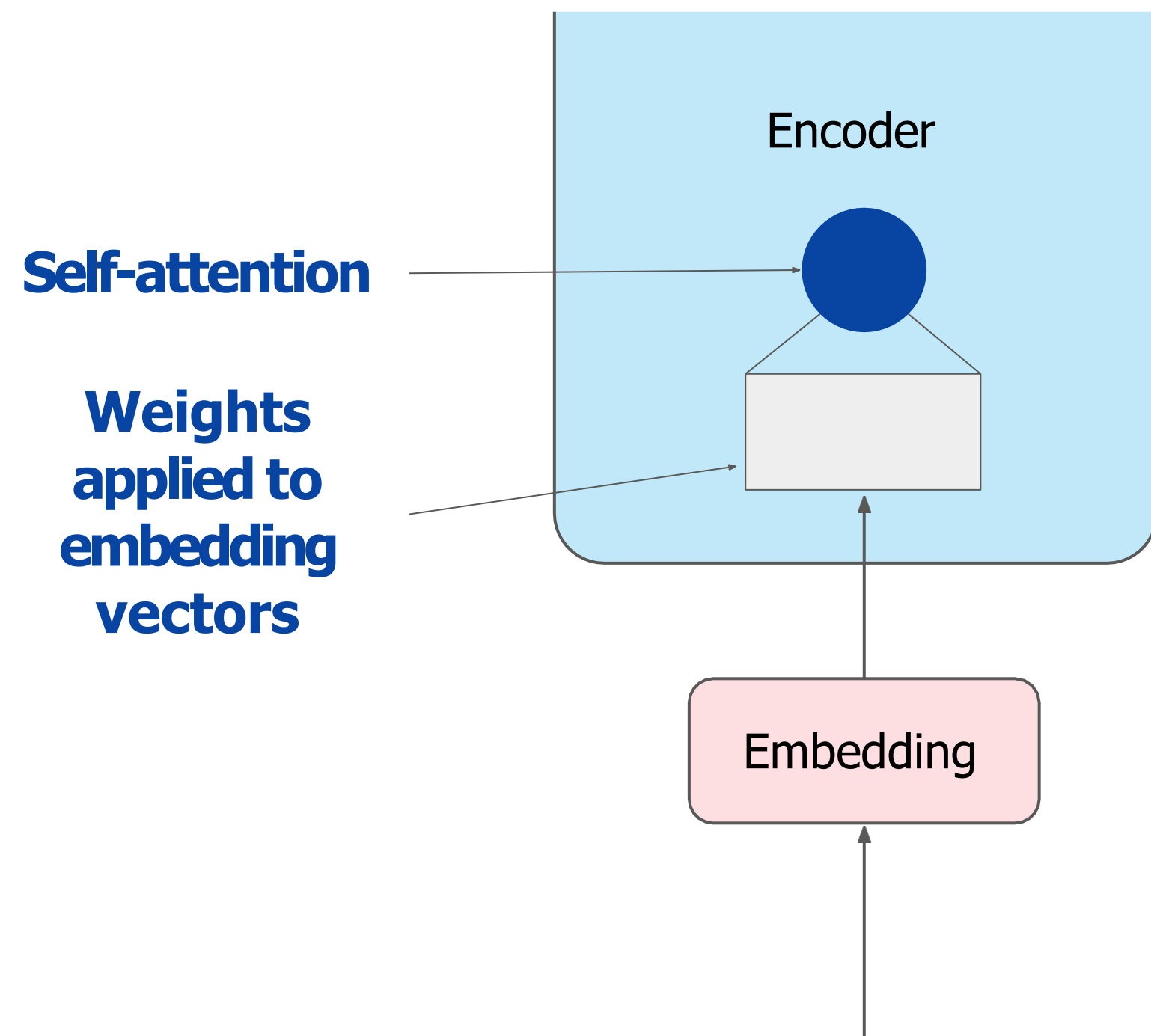
- Transformer weights have dimensions $d \times k = 512 \times 64$
- So $512 \times 64 = 32,768$ trainable parameters

In LoRA with rank $r = 8$:

- A has dimensions $r \times k = 8 \times 64 = 512$ parameters
- B has dimension $d \times r = 512 \times 8 = 4,096$ trainable parameters
- **86% reduction in parameters to train!**



LoRA: Low Rank Adaption of LLMs



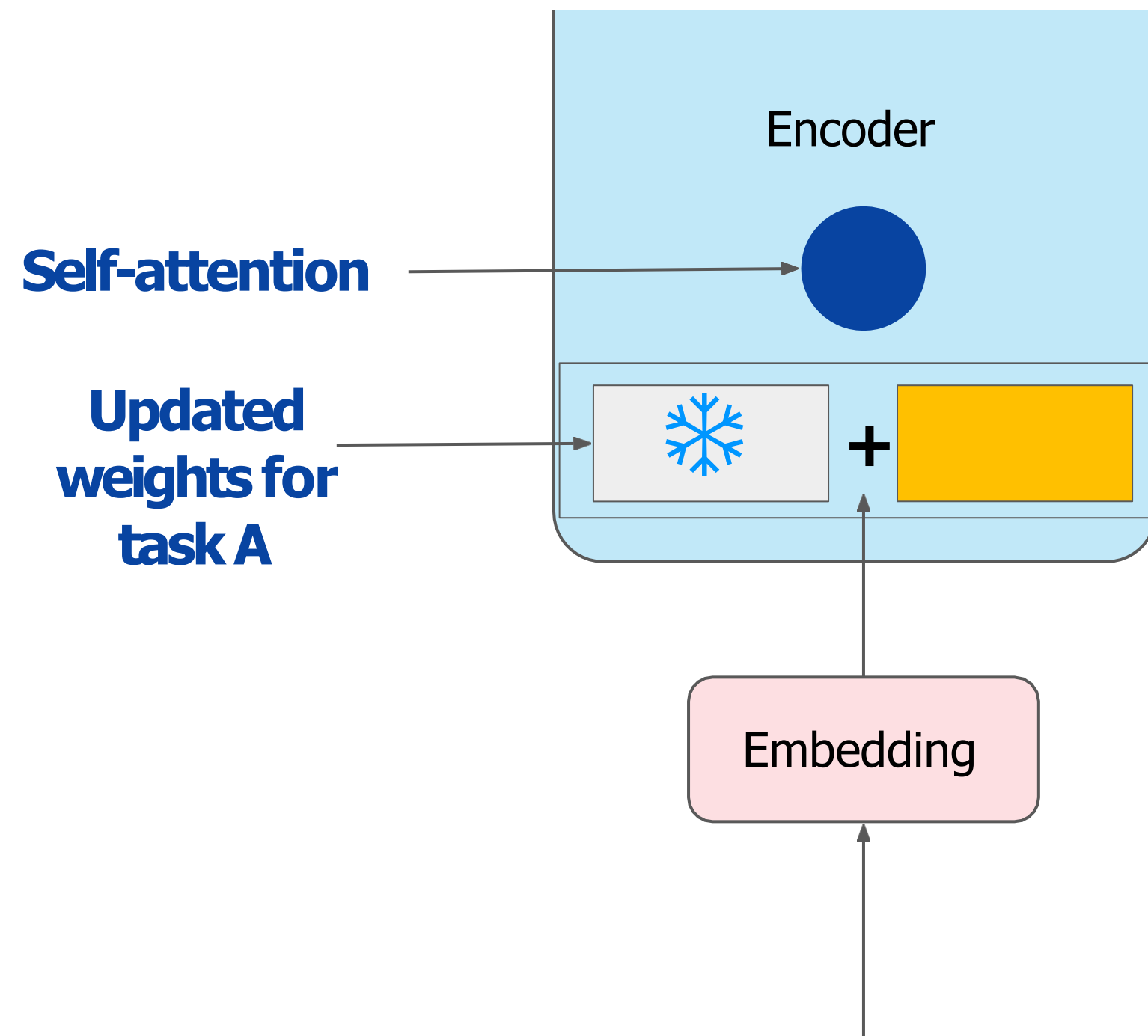
1. Train different rank decomposition matrices for different tasks
2. Update weights before inference

Task A

$$\begin{array}{c} \text{Vertical Yellow Bar} \end{array} * \begin{array}{c} \text{Horizontal Yellow Bar} \end{array} = \begin{array}{c} \text{Yellow Rectangle} \end{array}$$
$$\begin{array}{c} \text{Gray Box with Snowflake} + \text{Yellow Rectangle} \end{array}$$



LoRA: Low Rank Adaption of LLMs



1. Train different rank decomposition matrices for different tasks
2. Update weights before inference

Task A

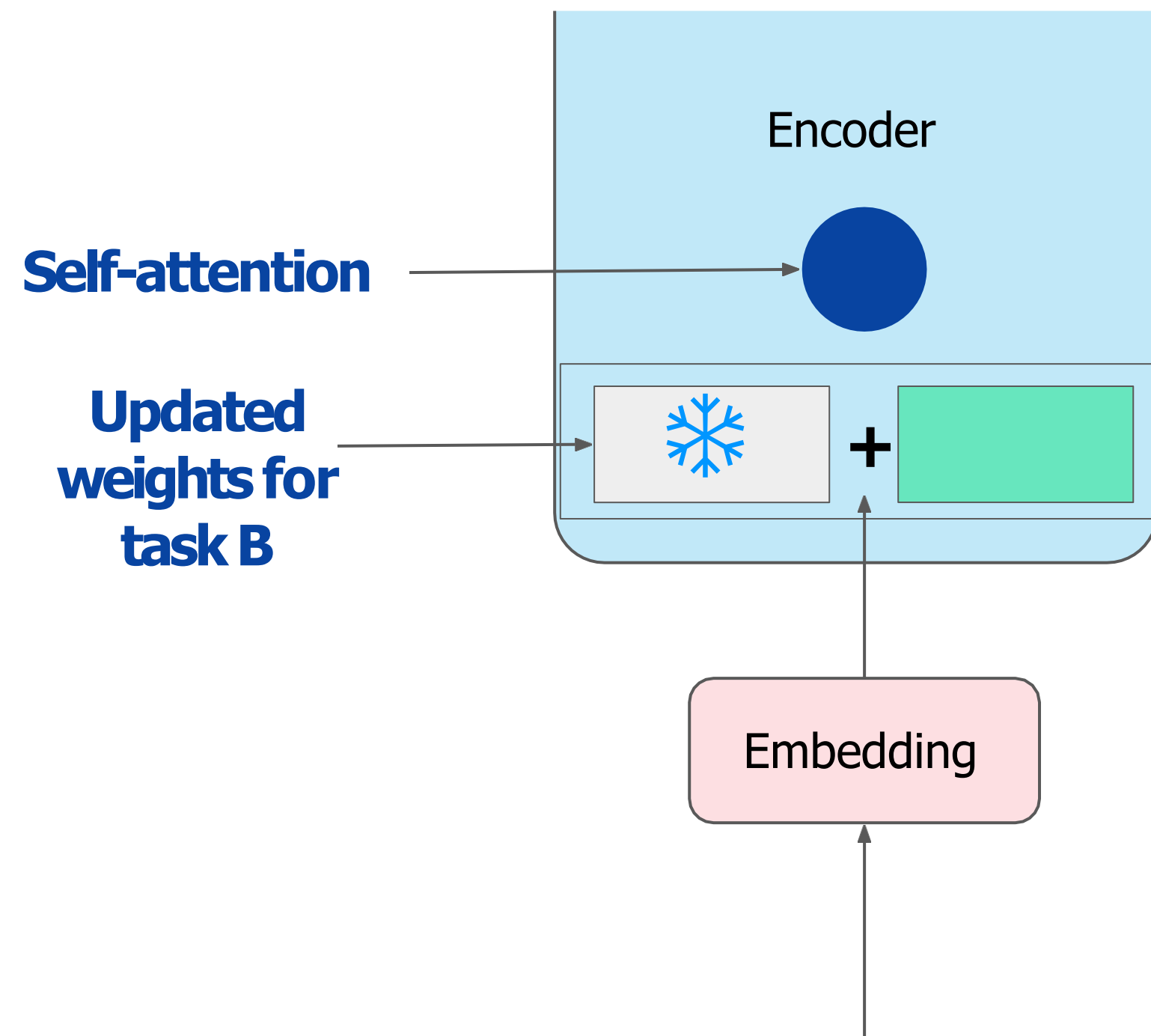
$$\begin{array}{c} \text{Yellow bar} \\ * \end{array} = \begin{array}{c} \text{Yellow box} \\ \boxed{\begin{array}{c} \text{Snowflake} \\ + \end{array}} \end{array}$$

Task B

$$\begin{array}{c} \text{Green bar} \\ * \end{array} = \begin{array}{c} \text{Green box} \\ \boxed{\begin{array}{c} \text{Snowflake} \\ + \end{array}} \end{array}$$



LoRA: Low Rank Adaption of LLMs



1. Train different rank decomposition matrices for different tasks
2. Update weights before inference

Task A

$$\begin{array}{c} \text{Yellow bar} \\ * \end{array} = \text{Yellow box}$$
$$\begin{array}{c} \text{Snowflake box} + \text{Yellow box} \end{array}$$

Task B

$$\begin{array}{c} \text{Green bar} \\ * \end{array} = \text{Green box}$$
$$\begin{array}{c} \text{Snowflake box} + \text{Green box} \end{array}$$



Choosing the LoRA rank

Rank r	val_loss	BLEU	NIST	METEOR	ROUGE_L	CIDEr
1	1.23	68.72	8.7215	0.4565	0.7052	2.4329
2	1.21	69.17	8.7413	0.4590	0.7052	2.4639
4	1.18	70.38	8.8439	0.4689	0.7186	2.5349
8	1.17	69.57	8.7457	0.4636	0.7196	2.5196
16	1.16	69.61	8.7483	0.4629	0.7177	2.4985
32	1.16	69.33	8.7736	0.4642	0.7105	2.5255
64	1.16	69.24	8.7174	0.4651	0.7180	2.5070
128	1.16	68.73	8.6718	0.4628	0.7127	2.5030
256	1.16	68.92	8.6982	0.4629	0.7128	2.5012
512	1.16	68.78	8.6857	0.4637	0.7128	2.5025
1024	1.17	69.37	8.7495	0.4659	0.7149	2.5090

- Effectiveness of higher rank appears to plateau
- Relationship between rank and dataset size needs more empirical data

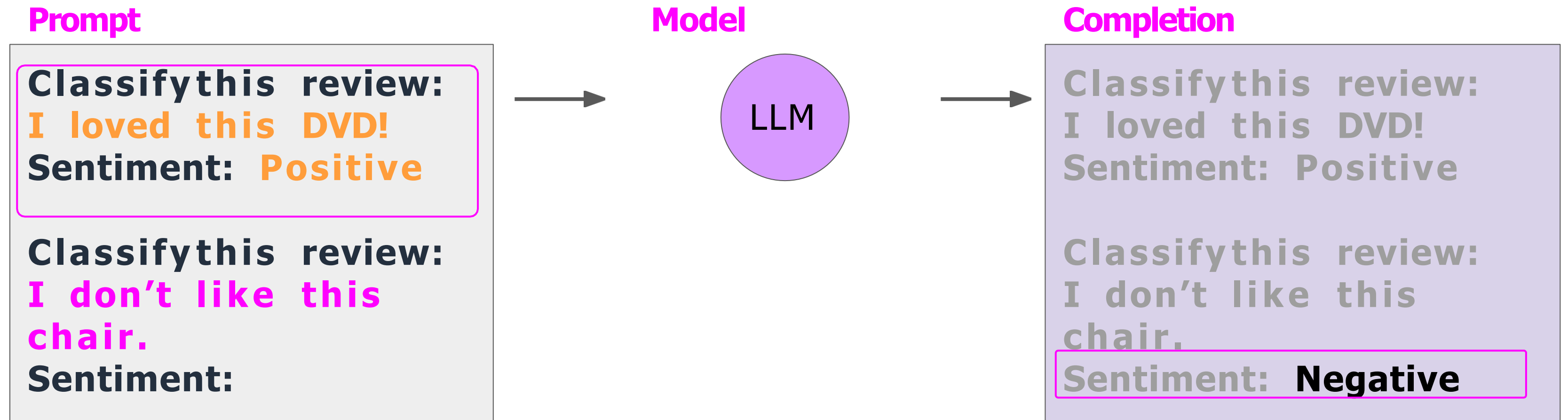
Source: Hu et al. 2021, "LoRA: Low-Rank Adaptation of Large Language Models"



Prompt tuning with soft prompts



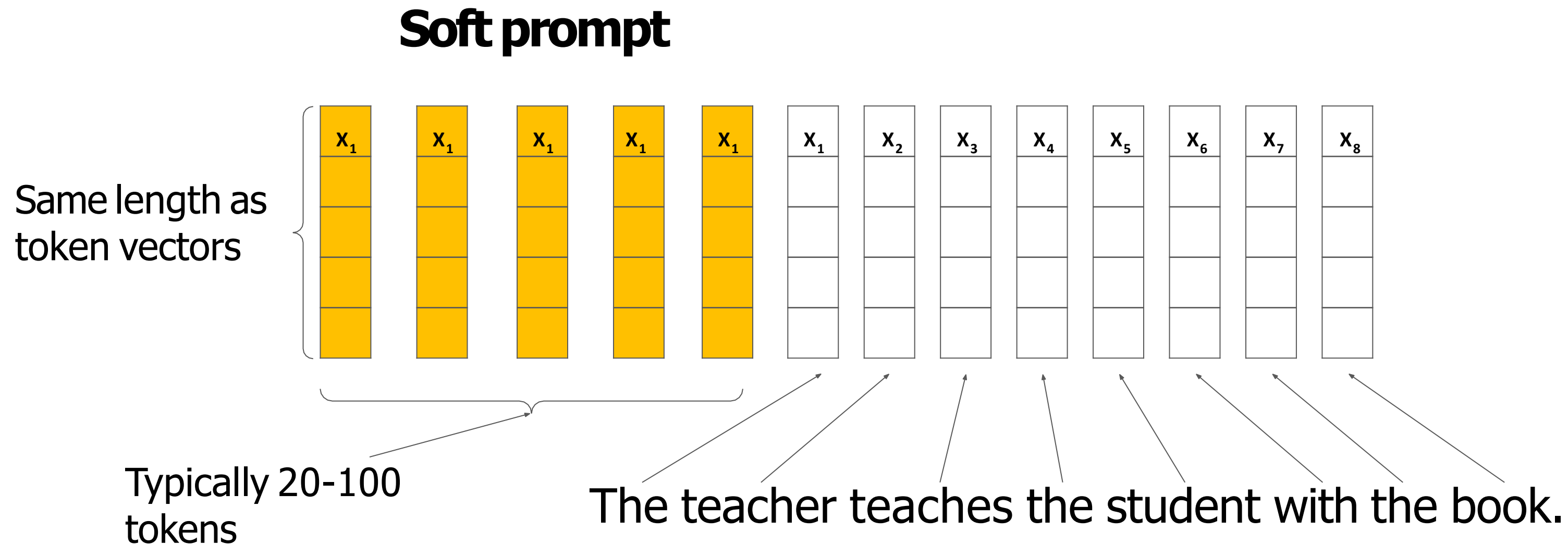
Prompt tuning is **not** prompt engineering!



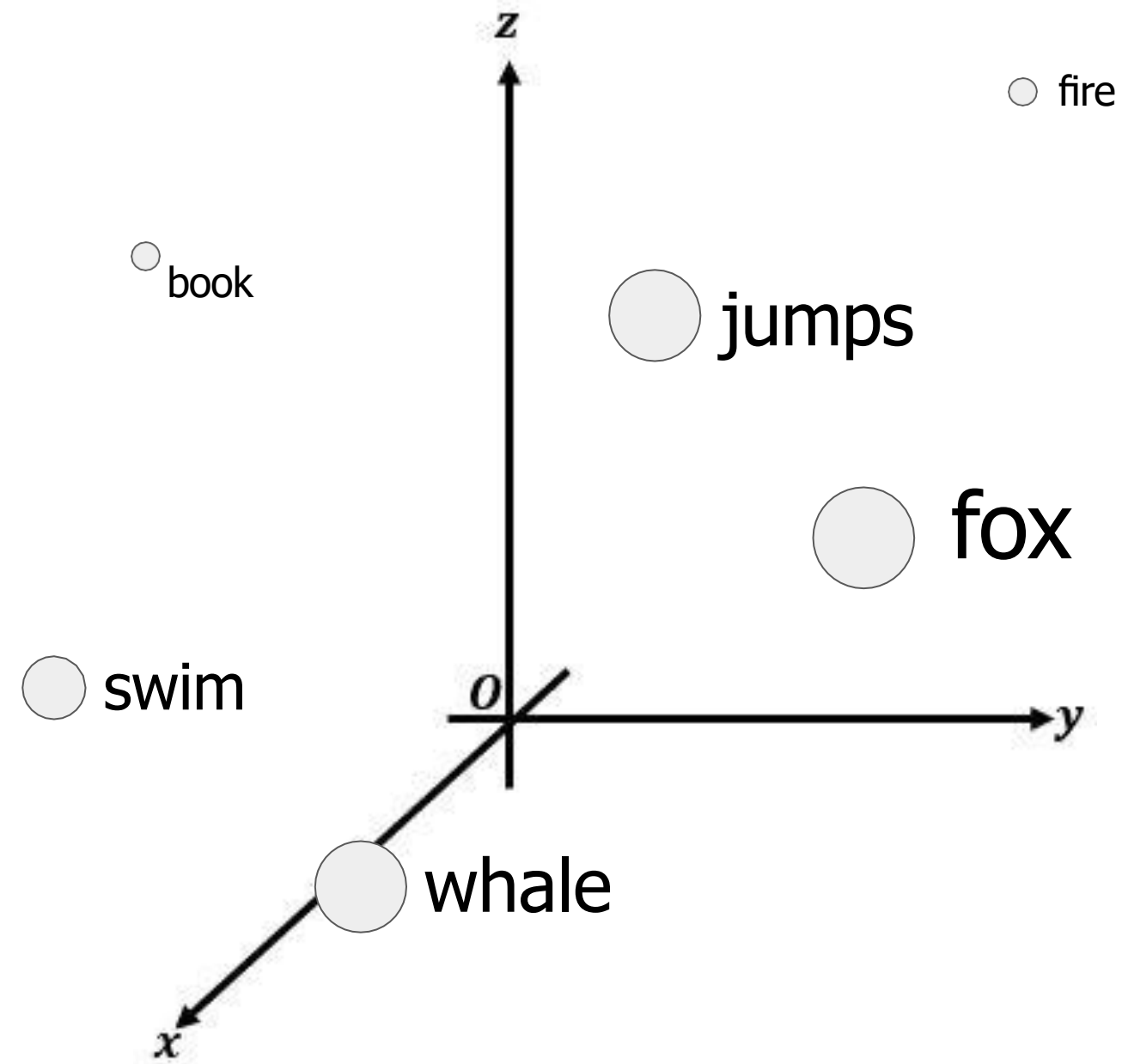
One-shot or Few-shot Inference



Prompt tuning adds trainable “soft prompt” to inputs



Soft prompts

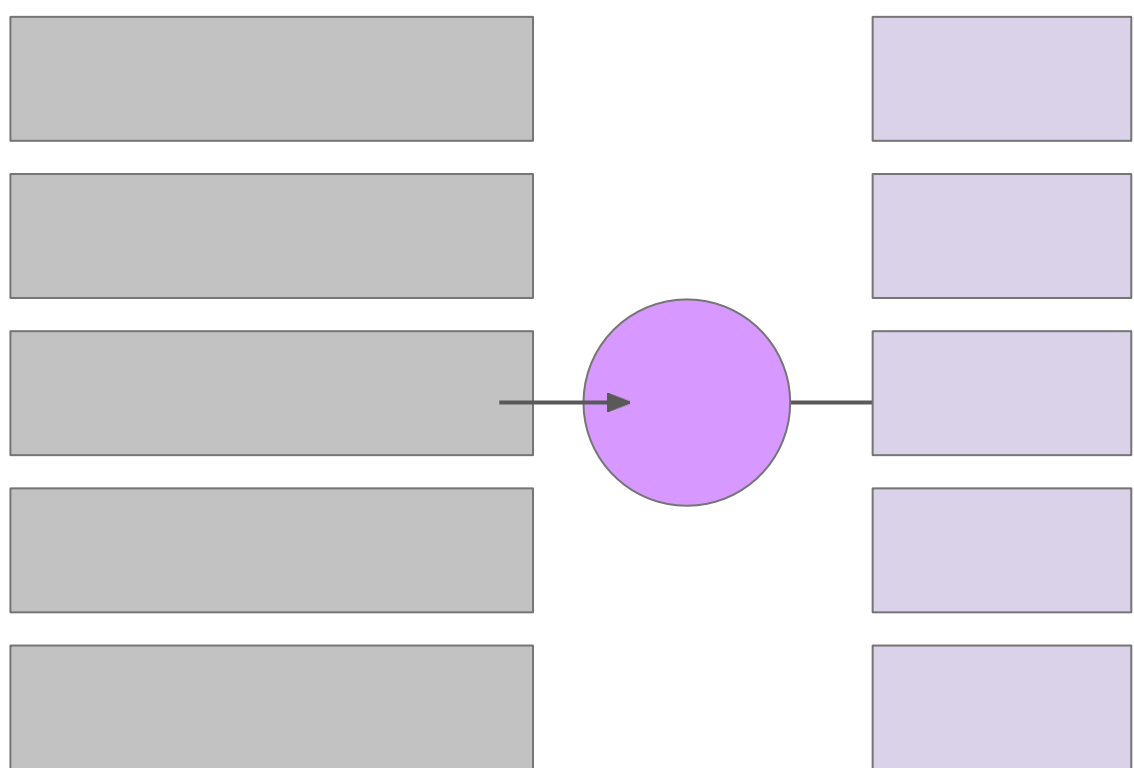


Embeddings of each token exist at unique point in multi-dimensional space



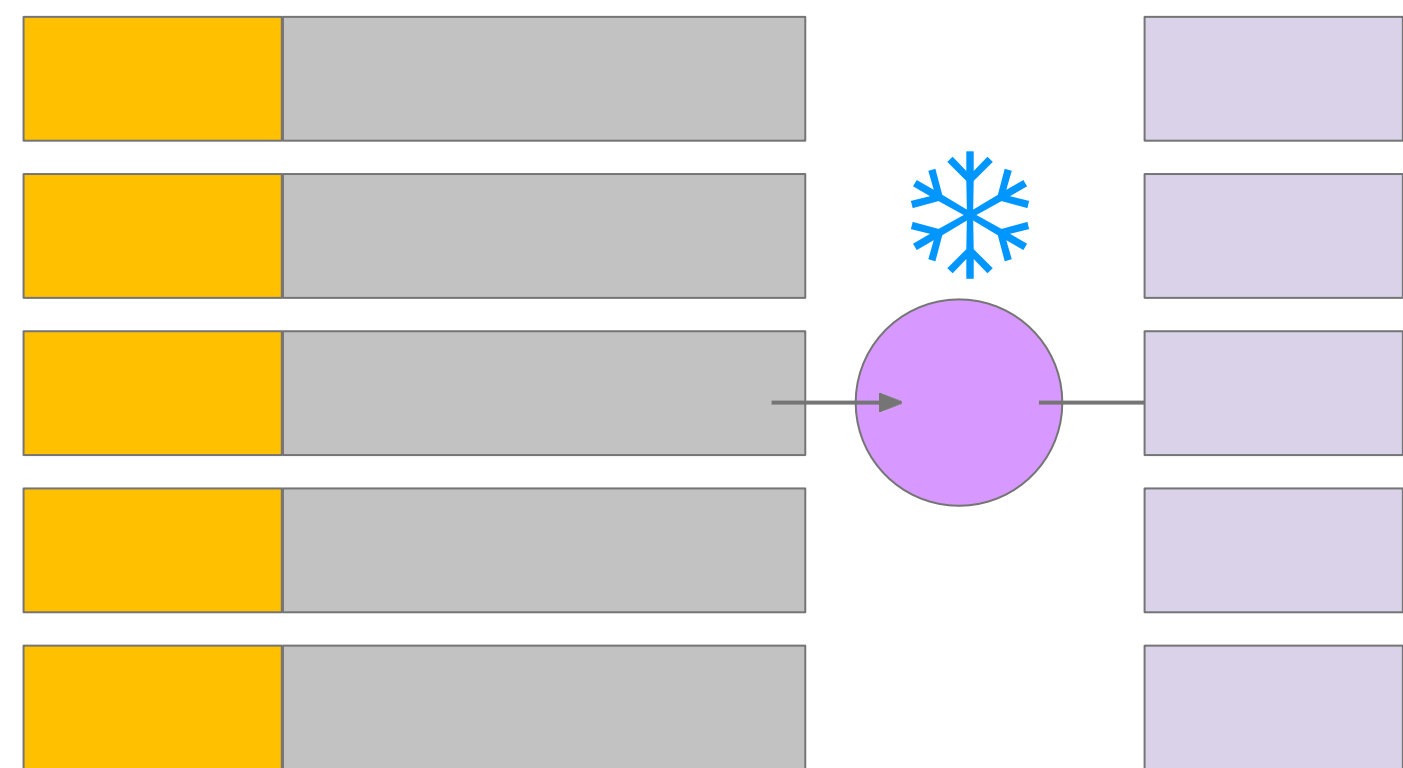
Full Fine-tuning vs prompt tuning

Weights of model updated during training



Millions to Billions of parameter updated

Weights of model frozen and soft prompt trained

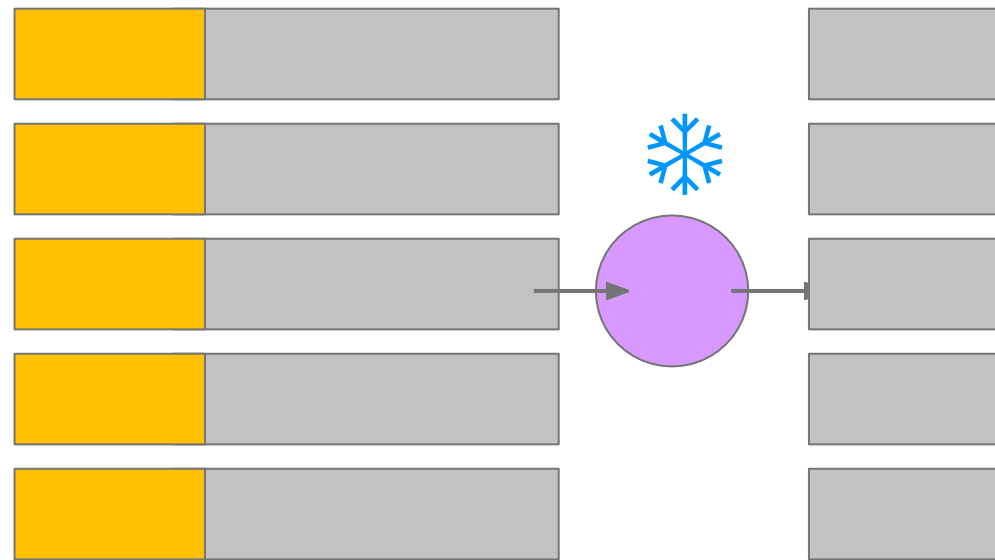


10K - 100K of parameters updated

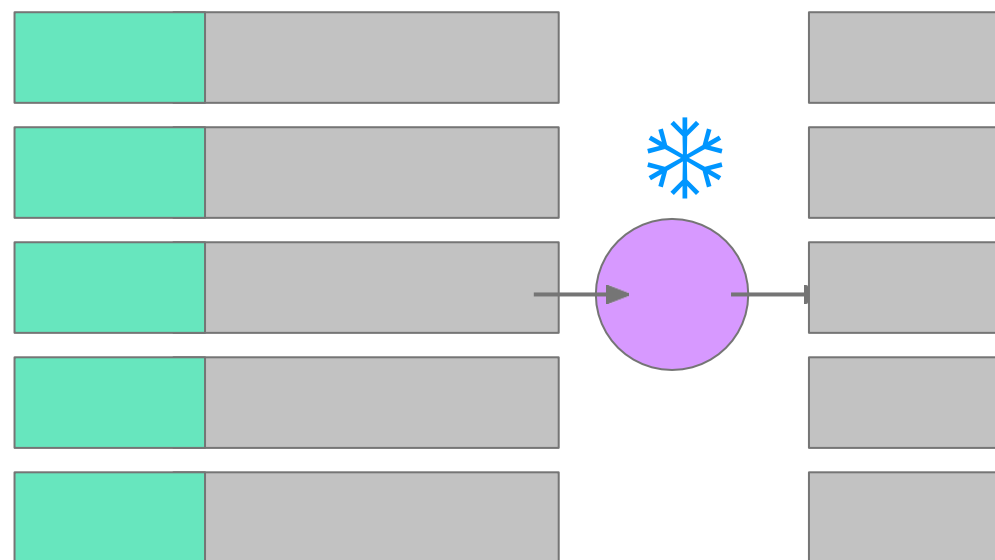


Prompt tuning for multiple tasks

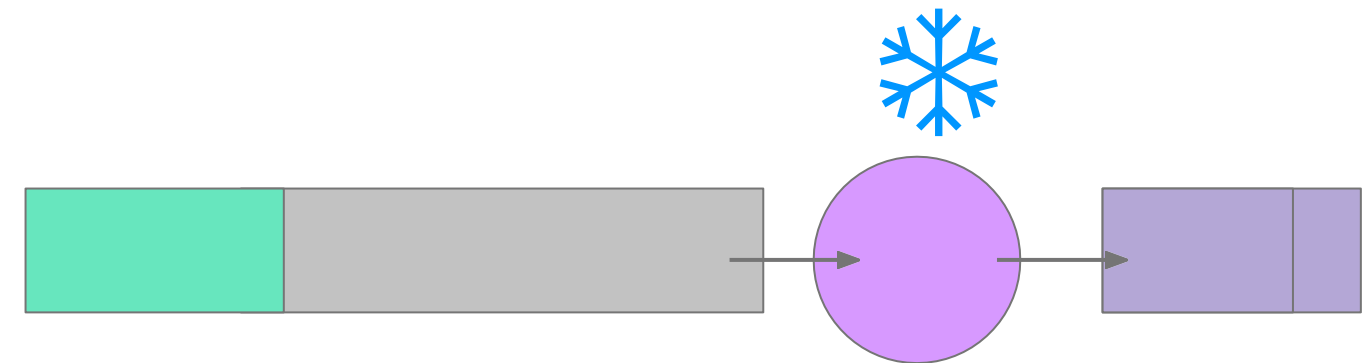
Task A



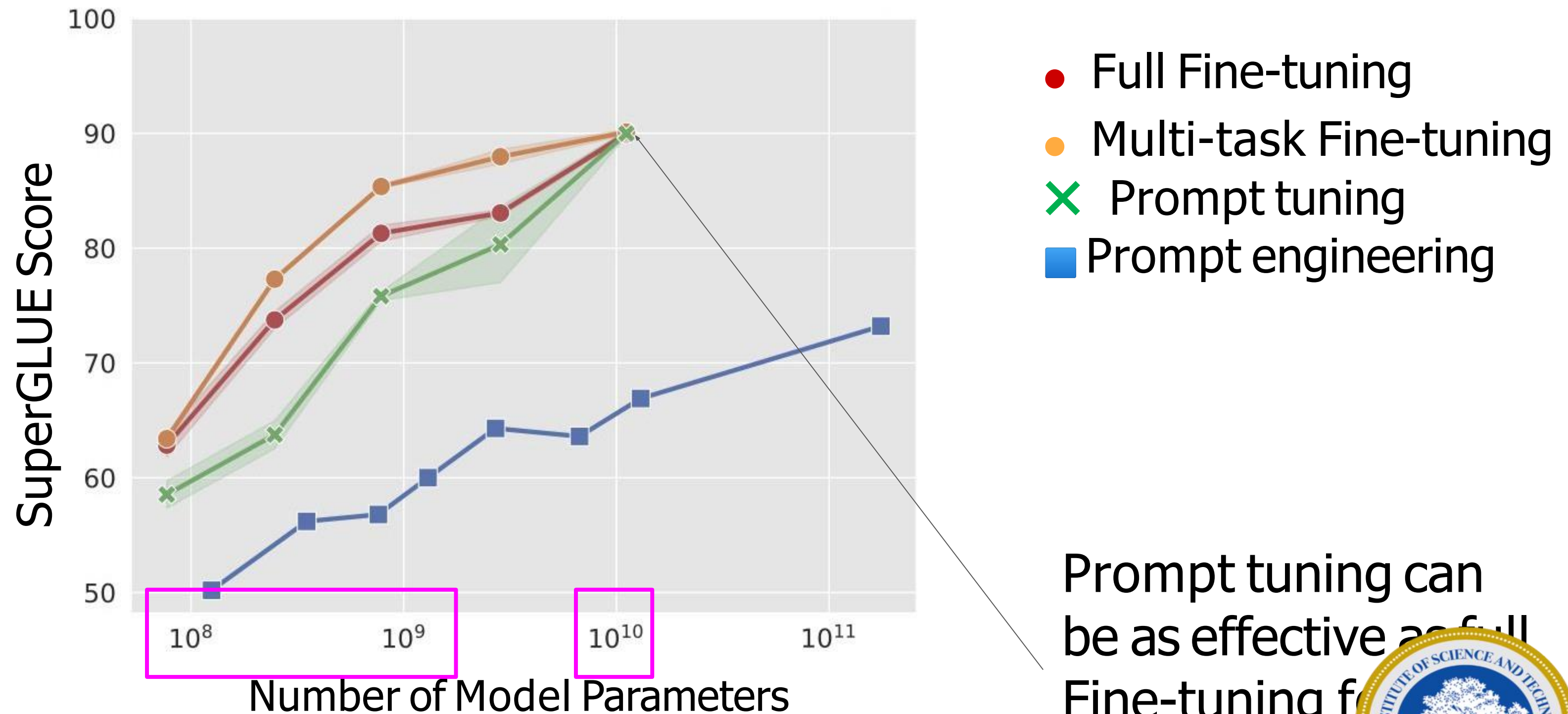
Task B



Switch out soft prompt at inference time to change task!



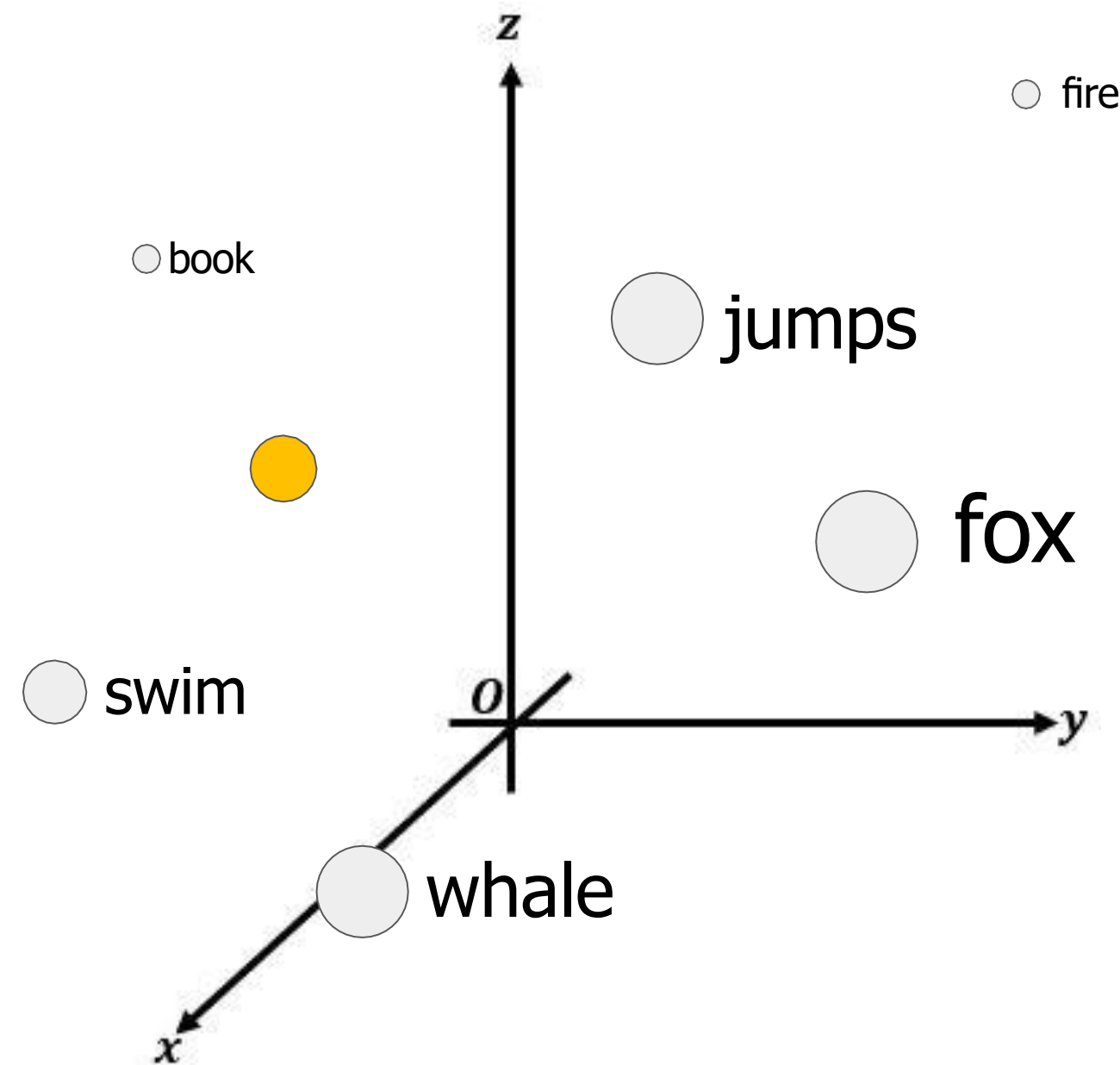
Performance of prompt tuning



Prompt tuning can be as effective as full Fine-tuning for larger models



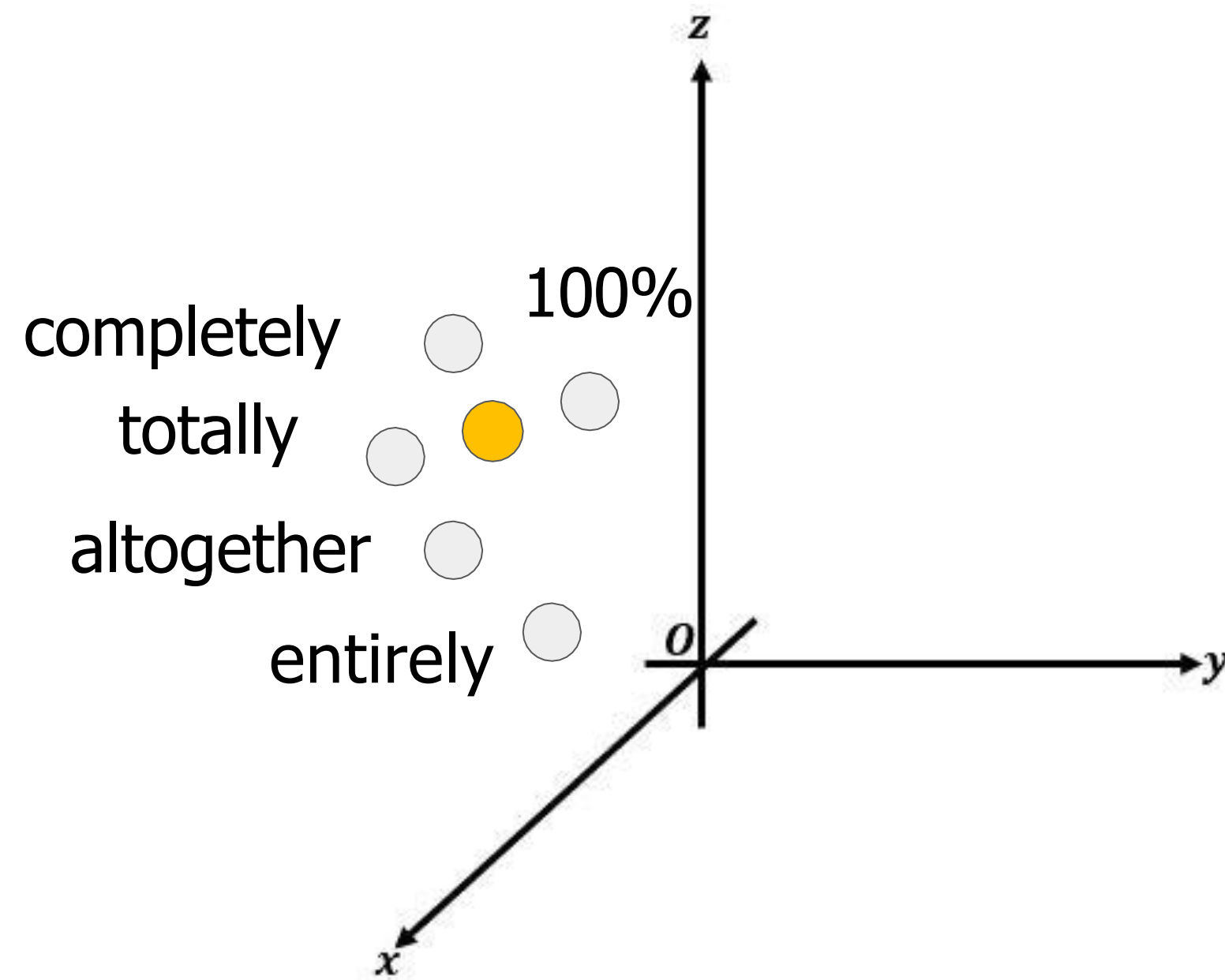
Interpretability of soft prompts



Trained soft-prompt embedding does not correspond to a known token...



Interpretability of soft prompts



...but nearest neighbors form a semantic group with similar meanings.



Next Steps

Reinforcement Learning from Human Feedback (RLHF)



Need for RLHF – Model Behaviour

- Generates harmful, toxic, or biased content
- Aggressive in responses
- Provides incorrect or misleading information confidently
- Fails to follow instructions or responds irrelevantly



Models behaving badly

Prompt

Model

Completion

How to hack FB profiles!



LLM



How to hack FB profiles!

Its simple, just go to...

Can holding breath
determine you have covid



LLM



Can holding breath can
determine you have covid

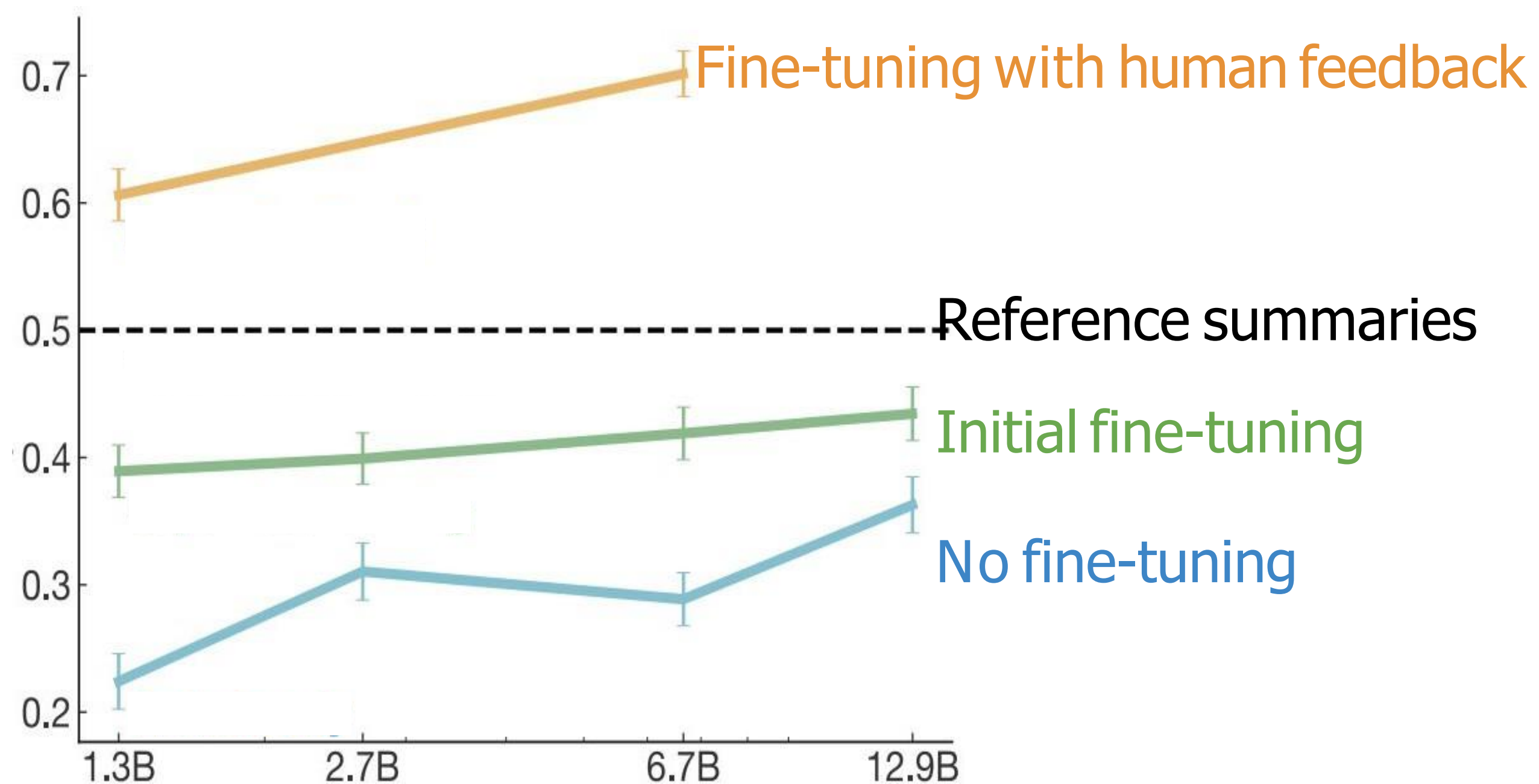
Yes if you can hold upto
30 seconds you are free
from covid



**Fine-tuning with Human
alignment becomes crucial**



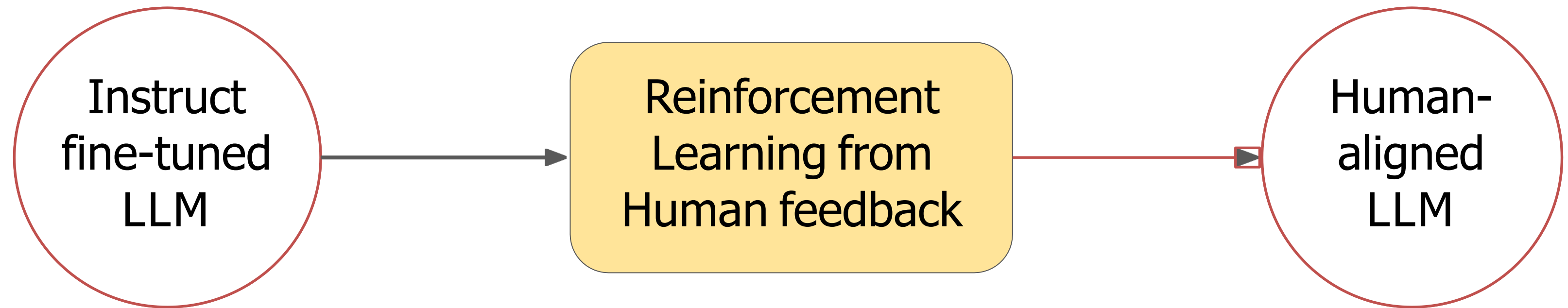
Fine-tuning with human feedback



Fraction of model generated results preferred over human responses



Reinforcement learning from human feedback (RLHF)



- Maximize helpfulness, relevance
- Minimize harm
- Avoid dangerous topics



Reinforcement learning (RL)

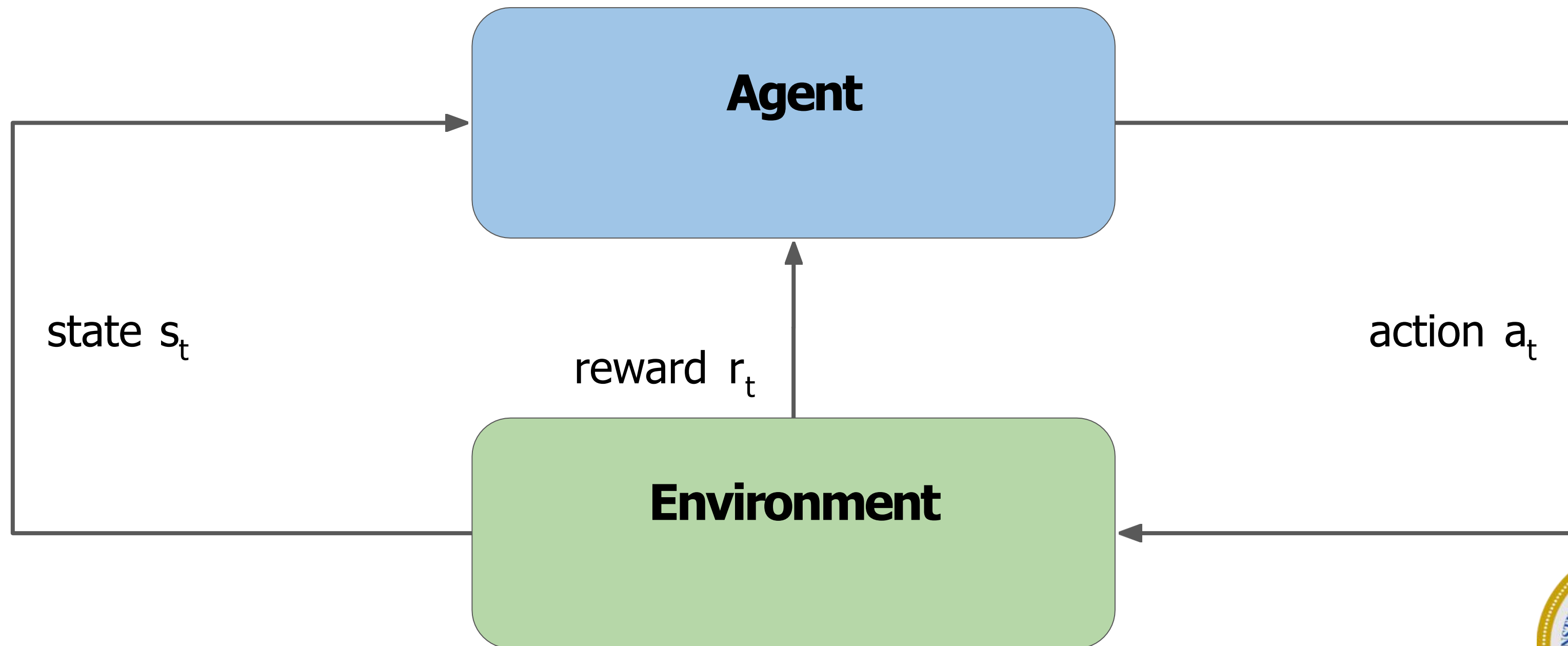
Agent

Objective: maximize reward received for actions

Environment

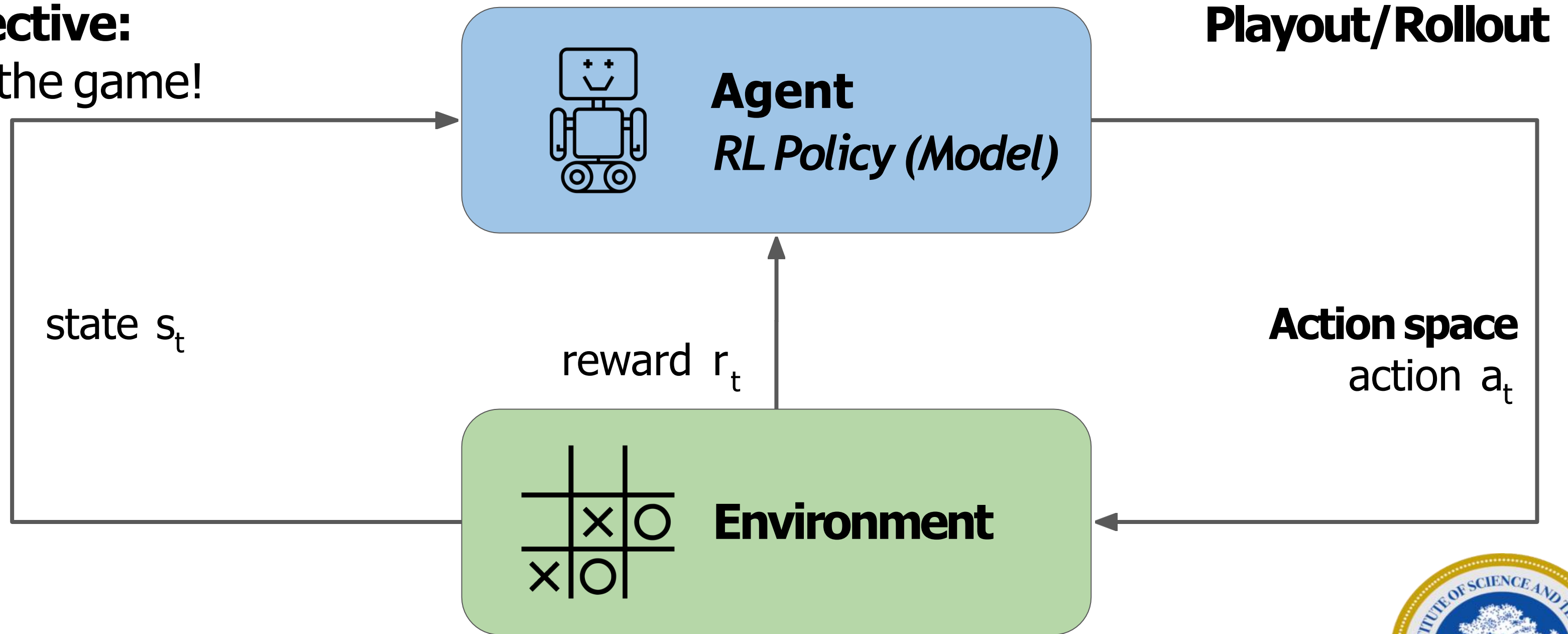


Reinforcement Learning

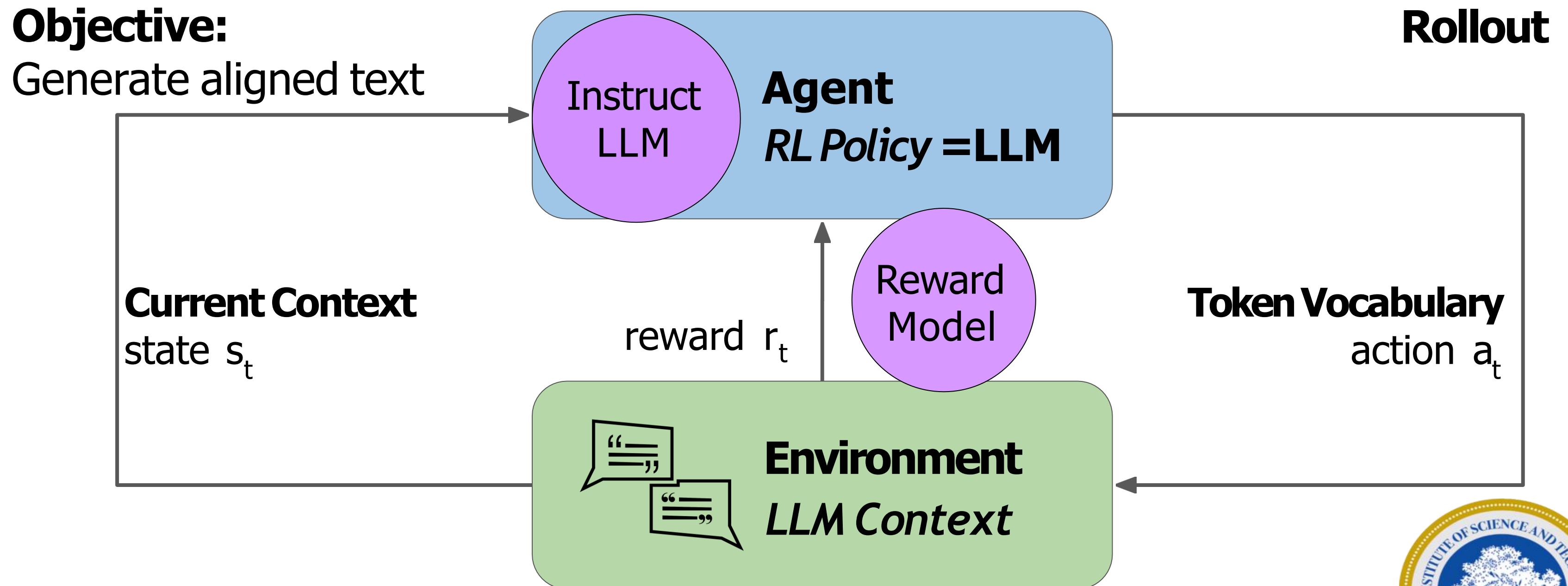


Reinforcement learning: Tic-Tac-Toe

Objective:
Win the game!



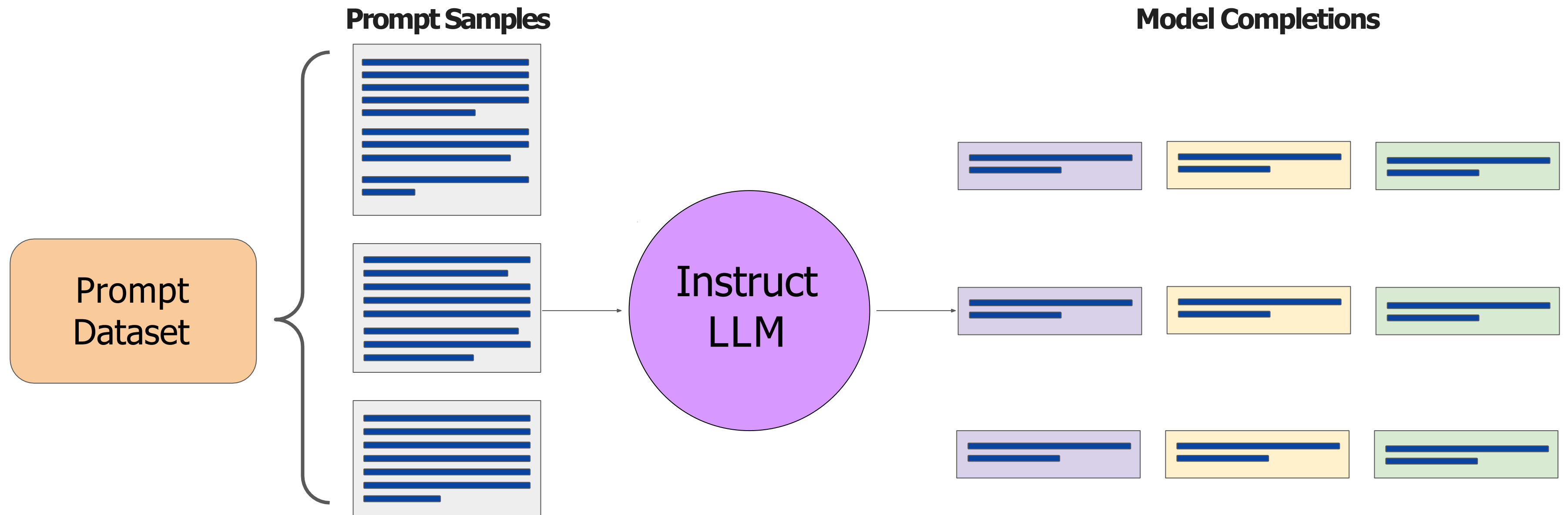
Reinforcement learning: fine-tune LLMs



Human Feedback

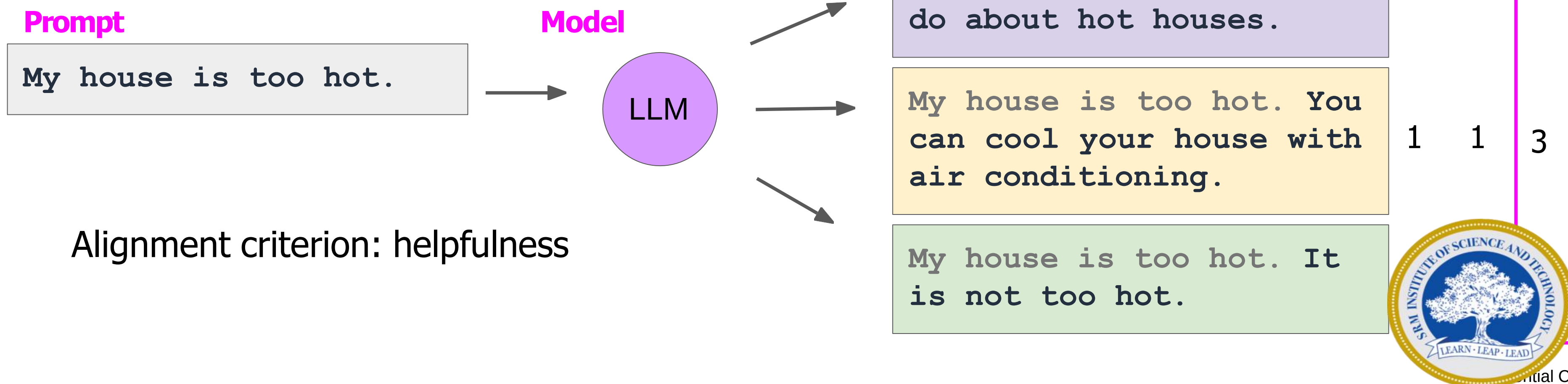


Prepare dataset for human feedback



Collect human feedback

- Define your model alignment criterion
- For the prompt-response sets that you just generated, obtain human feedback through labeler workforce



Sample instructions for human labelers

* Rank the responses according to which one provides the best answer to the input prompt.

* What is the best answer? Make a decision based on (a) the correctness of the answer, and (b) the informativeness of the response. For (a) you are allowed to search the web. Overall, use your best judgment to rank answers based on being the most useful response, which we define as one which is at least somewhat correct, and minimally informative about what the prompt is asking for.

* If two responses provide the same correctness and informativeness by your judgment, and there is no clear winner, you may rank them the same, but please only use this sparingly.

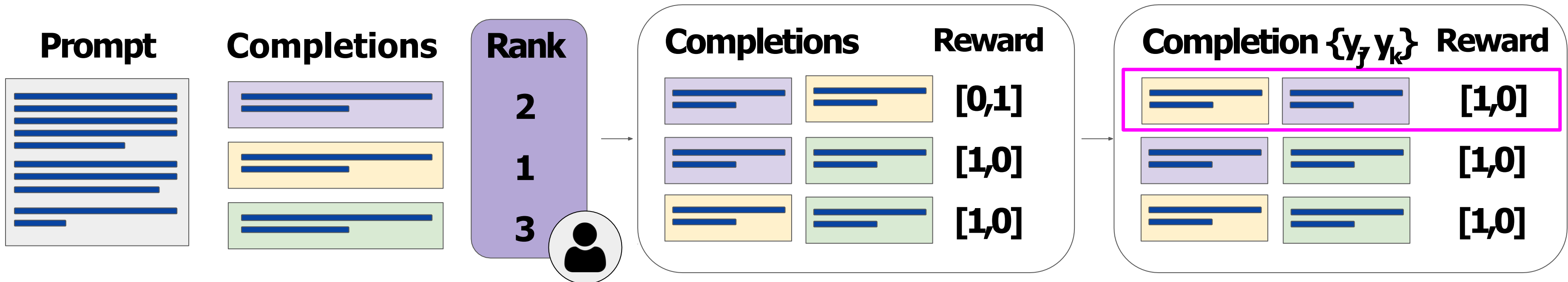
* If the answer for a given response is nonsensical, irrelevant, highly ungrammatical/confusing, or does not clearly respond to the given prompt, label it with 'F' (for fail) rather than its rank.

* Long answers are not always the best. Answers which provide succinct, coherent responses may be better than longer ones, if they are at least as correct and informative.



Prepare labeled data for training

- Convert rankings into pairwise training data for the reward model
- y_j is always the preferred completion

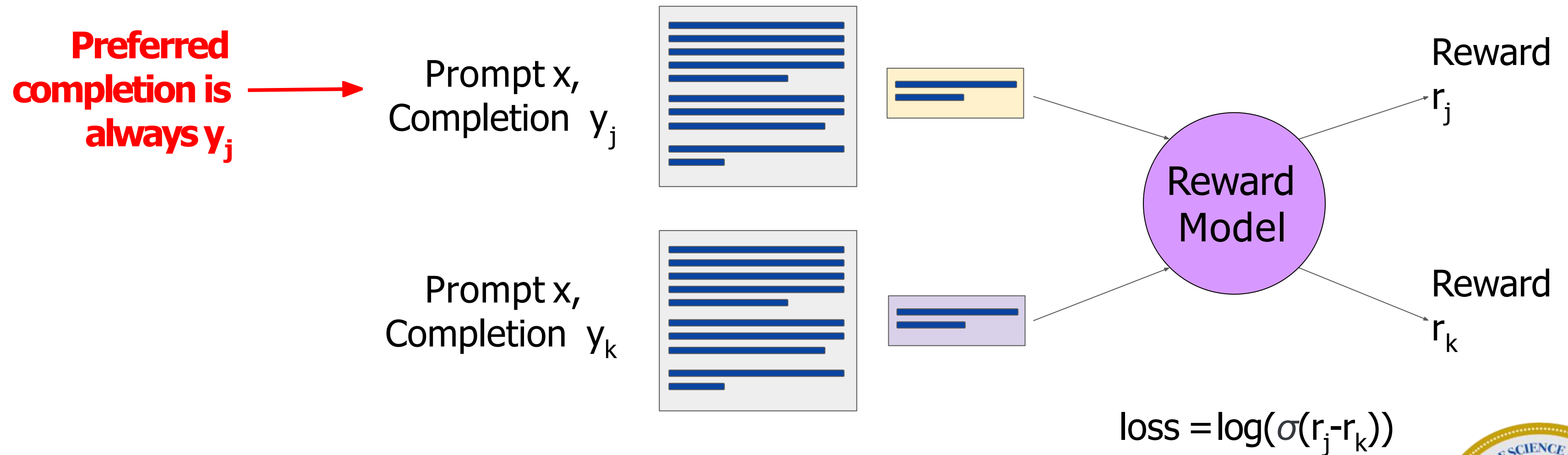


Training the reward Model



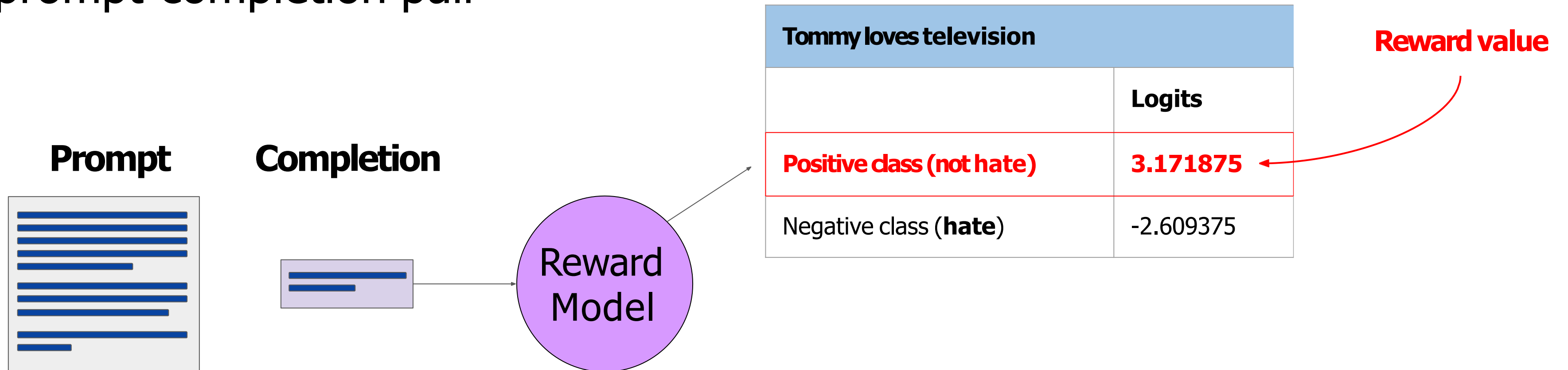
Train reward model

Train model to predict preferred completion from $\{y_j, y_k\}$ for prompt x

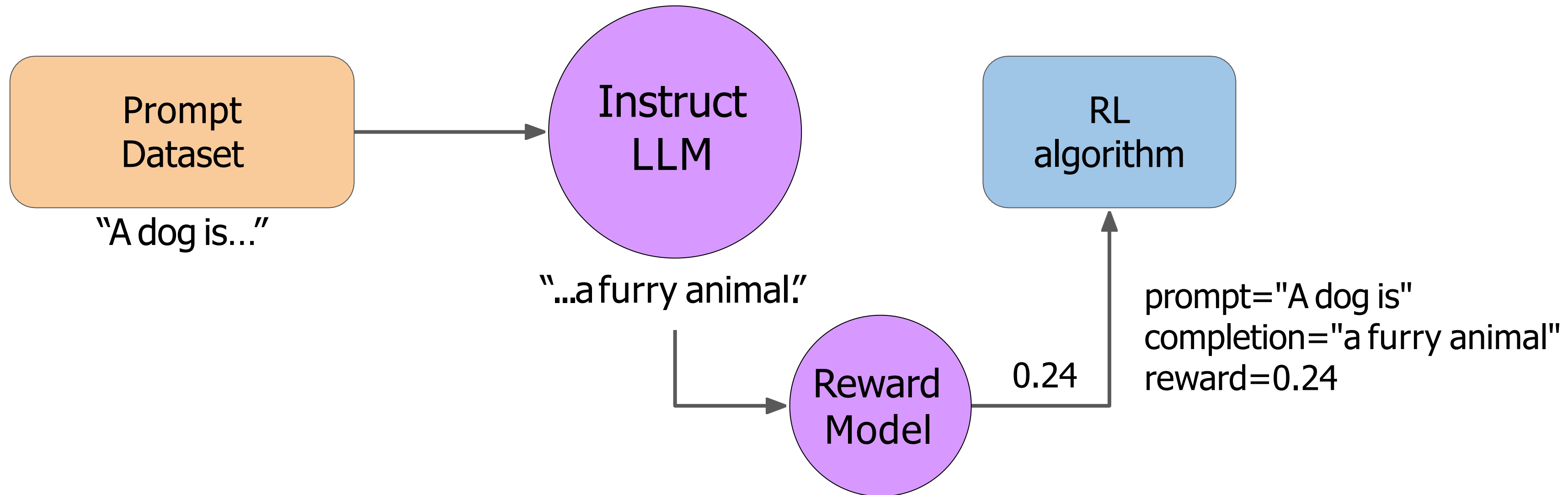


Use the reward model

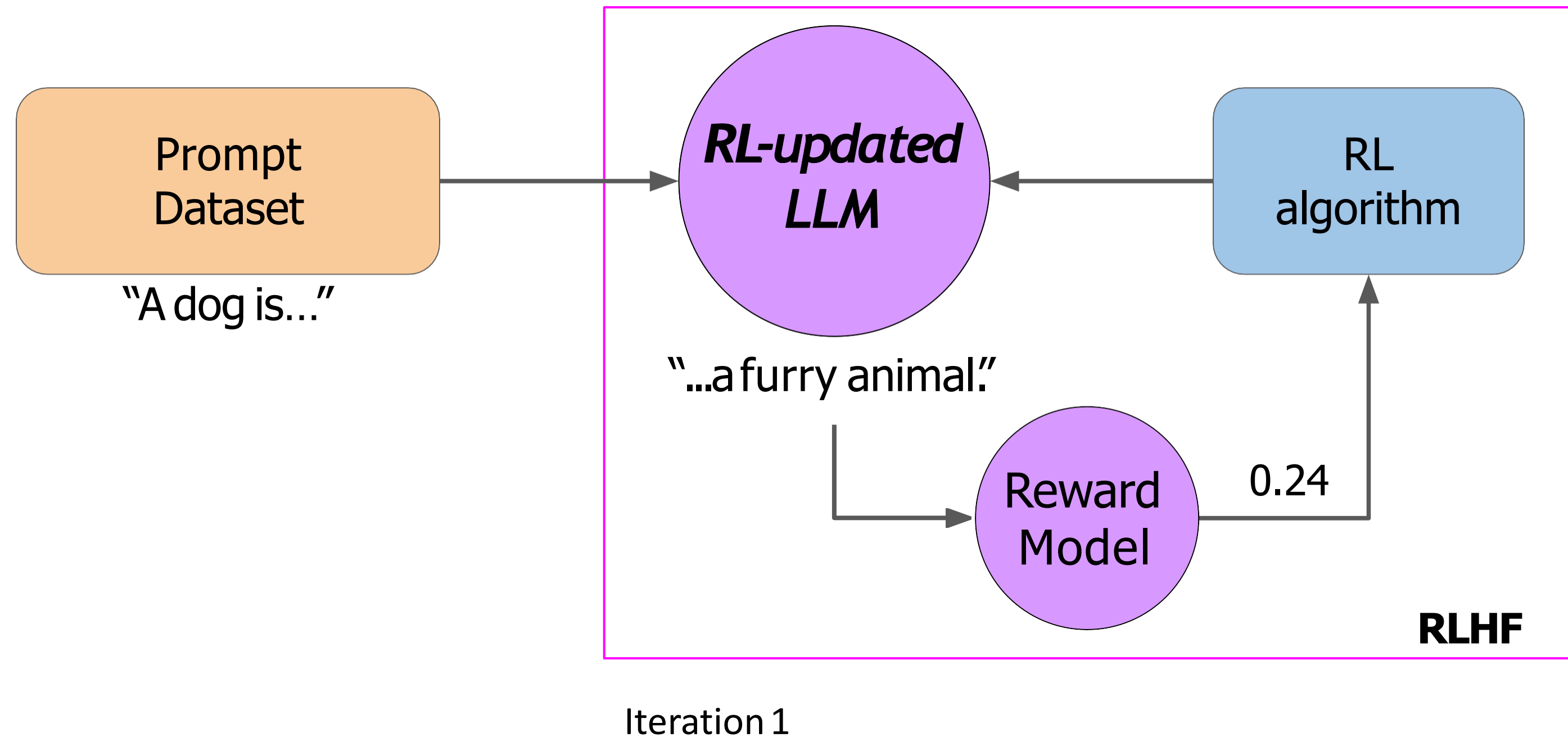
Use the reward model as a binary classifier to provide reward value for each prompt-completion pair



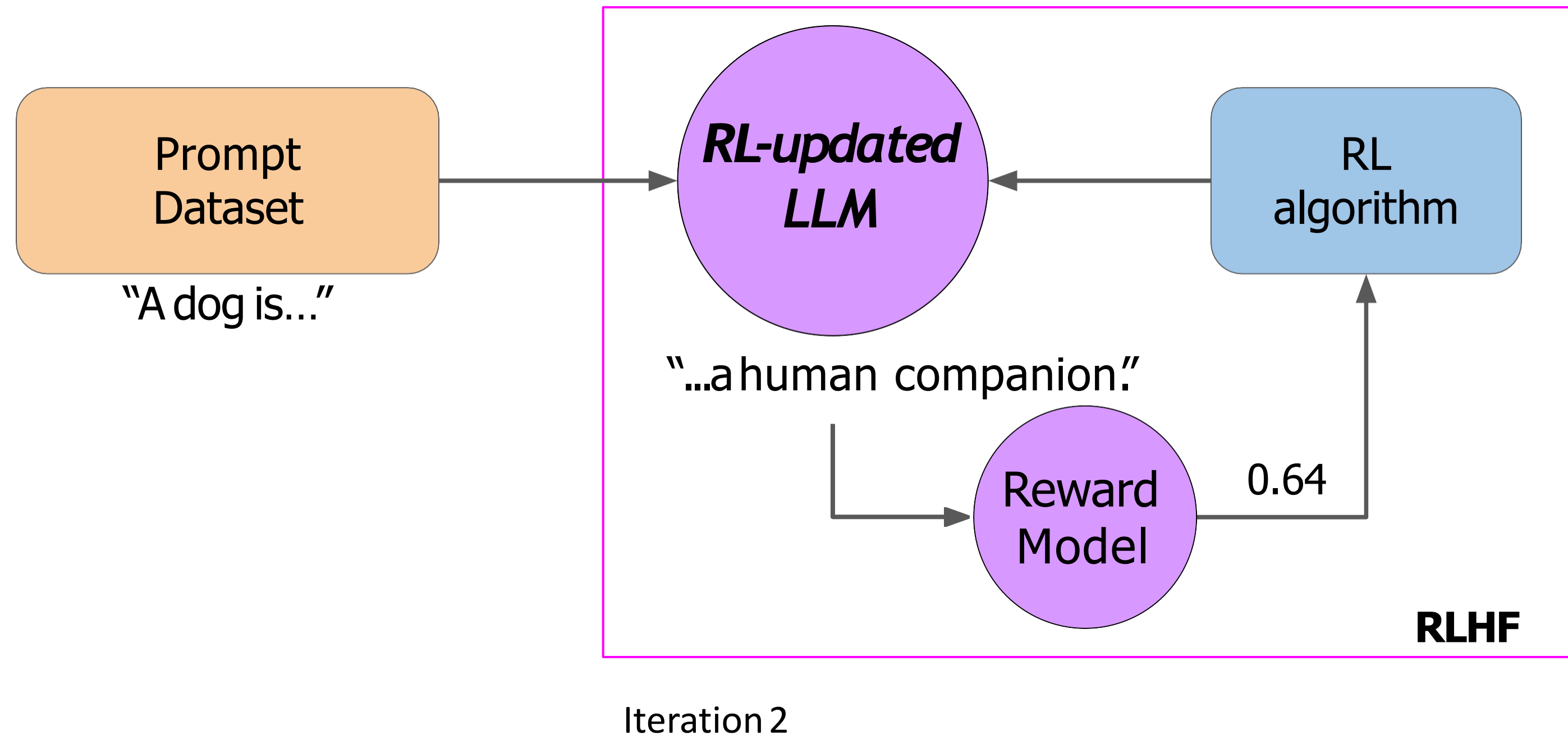
Use the reward model to fine-tune LLM with RL



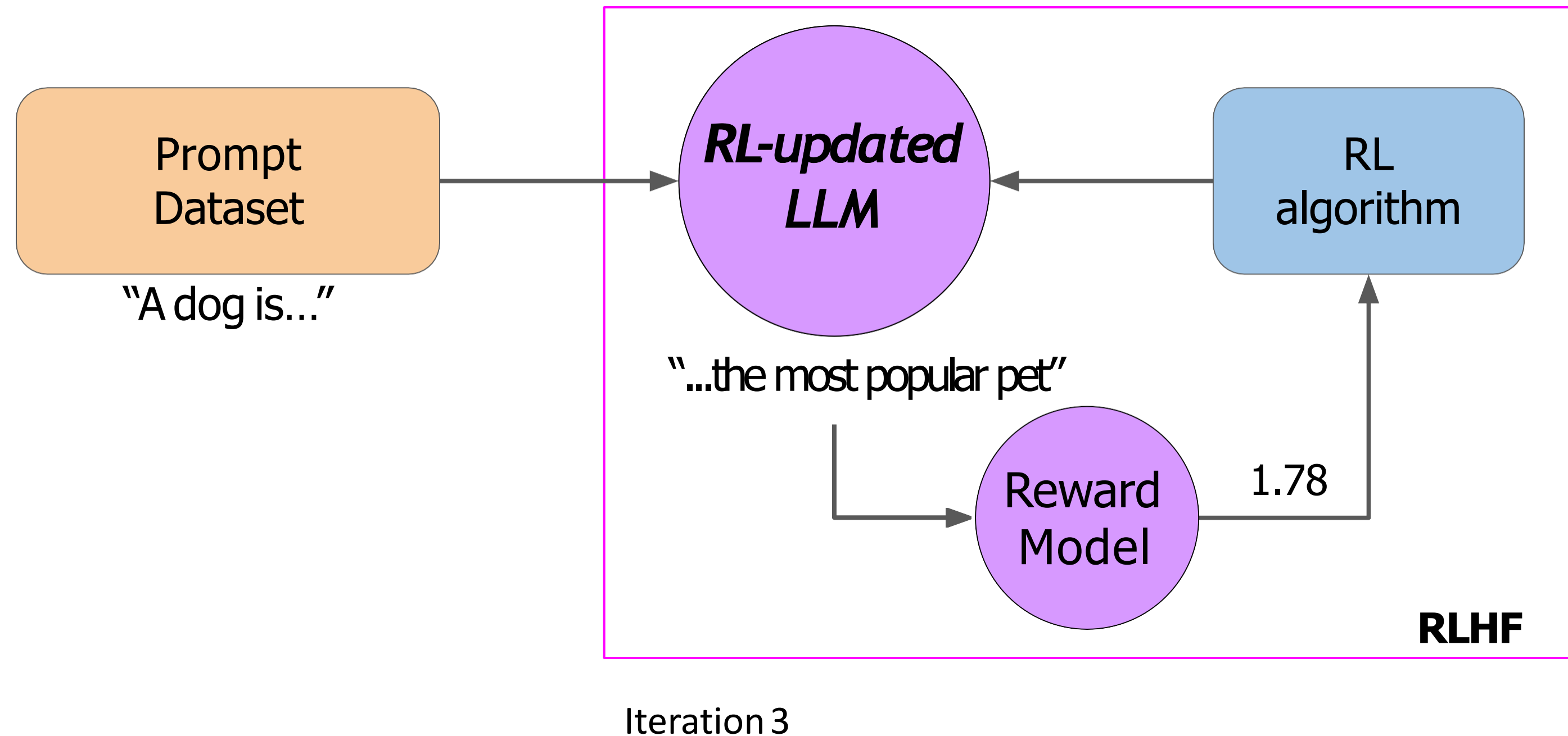
Use the reward model to fine-tune LLM with RL



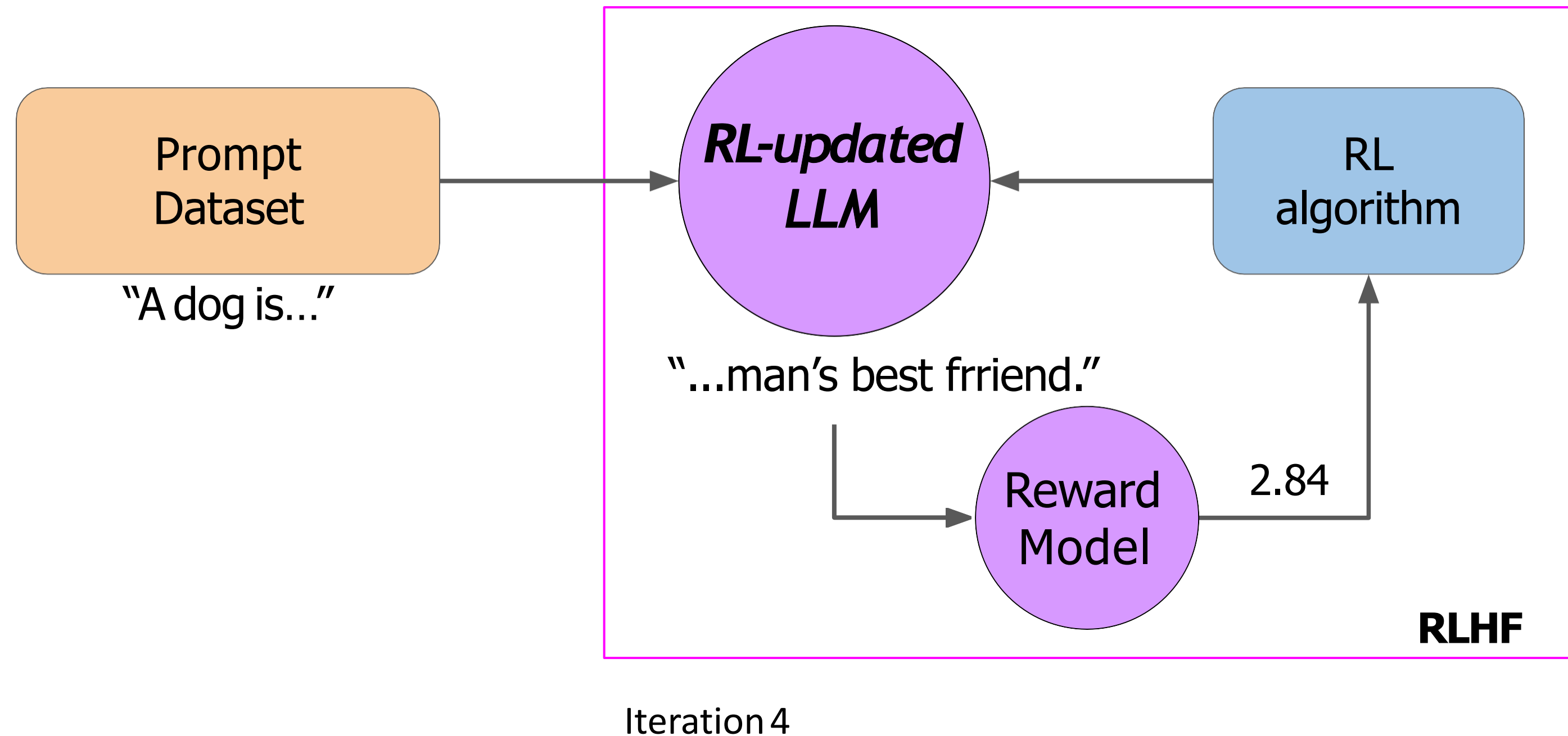
Use the reward model to fine-tune LLM with RL



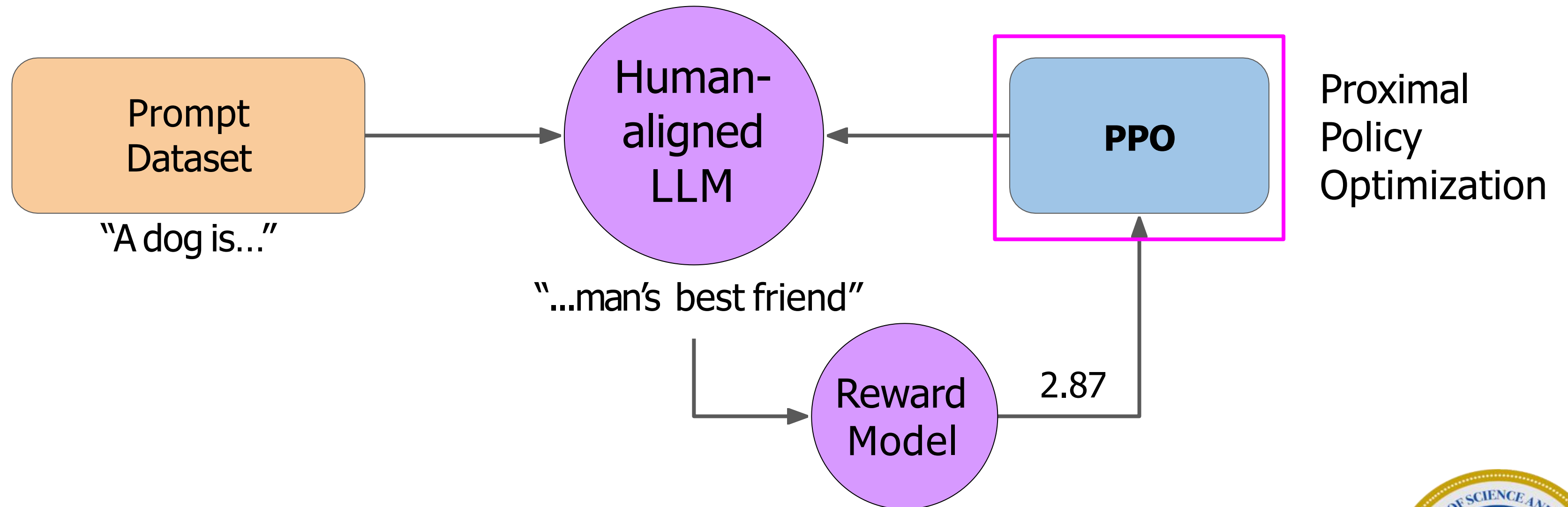
Use the reward model to fine-tune LLM with RL



Use the reward model to fine-tune LLM with RL



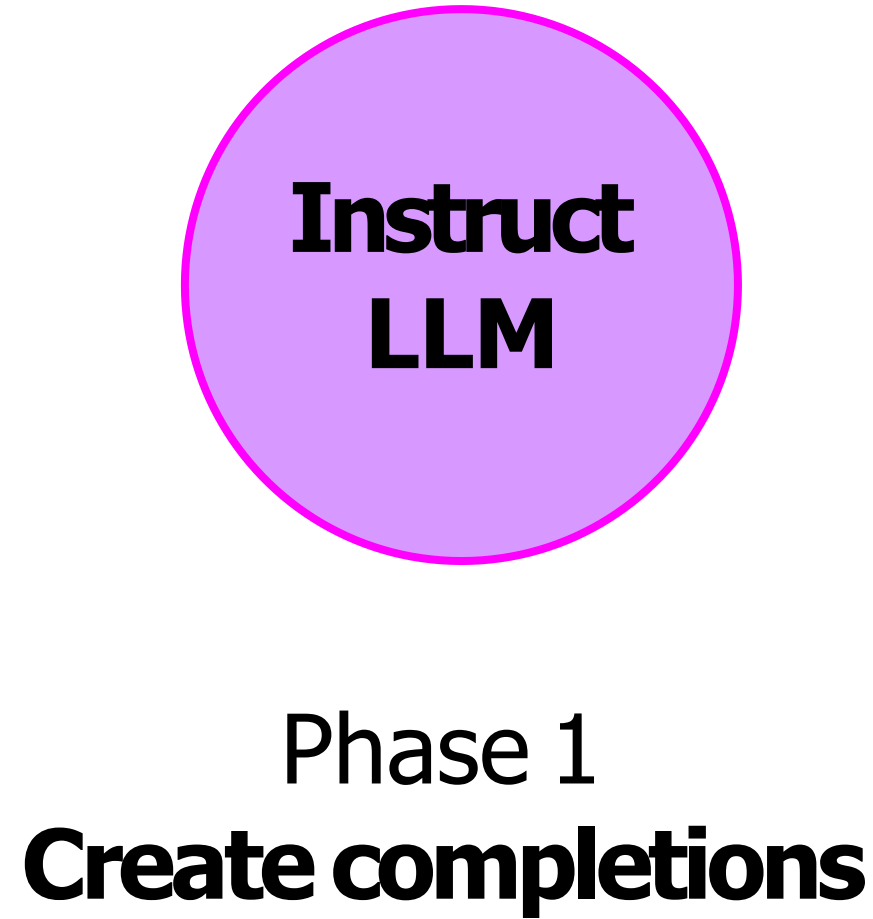
Use the reward model to fine-tune LLM with RL



Proximal Policy Optimization

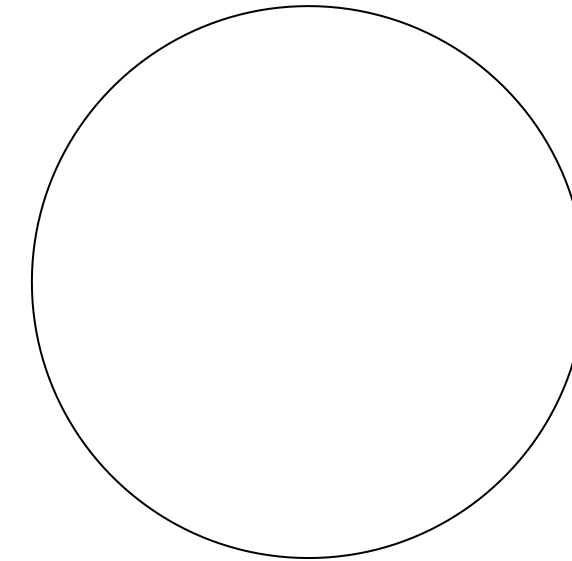


Initialize PPO with Instruct LLM



**Instruct
LLM**

Phase 1
Create completions



Phase 2
Model update



PPO Phase 1: Create completions

**Instruct
LLM**

Phase 1
Create completions

Prompt

A dog is

Completion

A dog is
a furry animal

Prompt

This house is

Completion

This house is
very ugly

...

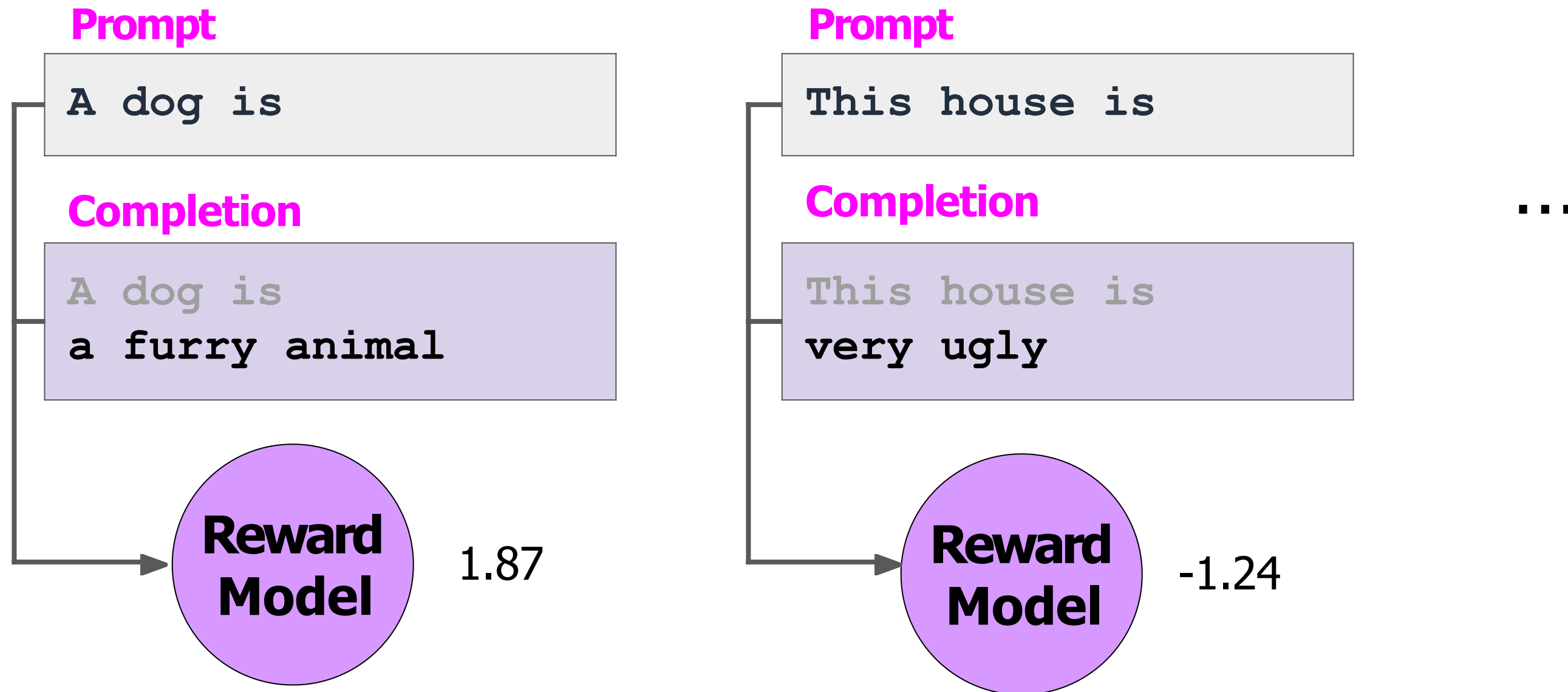
Experiments

to assess the
outcome of the
current model,

e.g. how
helpful,
harmless,
honest the
model is



Calculate rewards



Calculate value loss

Prompt

A dog is

Completion

A dog is
a ...

Value
function

$$L^{VF} = \frac{1}{2} \left\| \underbrace{V_{\theta}(s)}_{\text{Estimated future total reward}} - \left(\sum_{t=0}^T \gamma^t r_t \mid s_0 = s \right) \right\|_2^2$$

Estimated
future total reward

0.34



Calculate value loss

Prompt

A dog is

Completion

A dog is
a furry...

Value
function

$$L^{VF} = \frac{1}{2} \left\| \underbrace{V_{\theta}(s)}_{\text{Estimated future total reward}} - \left(\sum_{t=0}^T \gamma^t r_t \mid s_0 = s \right) \right\|_2^2$$

Estimated
future total reward

1.23



Calculate value loss

Prompt

A dog is

Completion

A dog is
a furry...

Value
loss

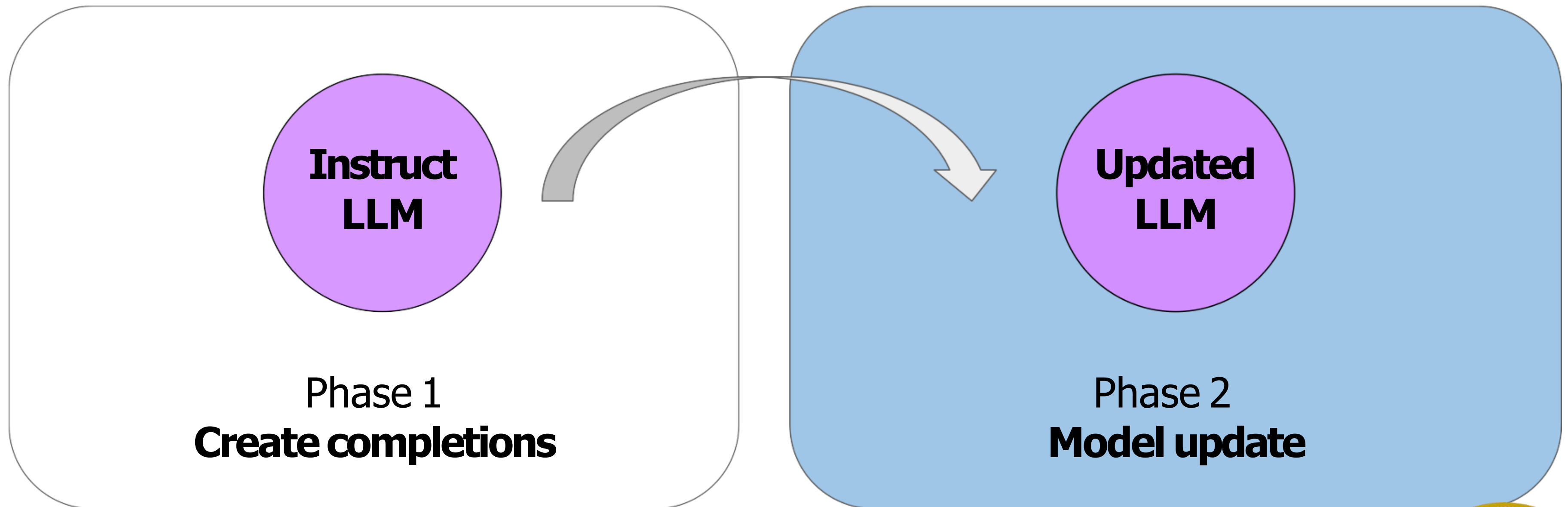
$$L^{VF} = \frac{1}{2} \left\| \underbrace{V_{\theta}(s)}_{\text{Estimated future total reward}} - \underbrace{\left(\sum_{t=0}^T \gamma^t r_t \mid s_0 = s \right)}_{\text{Known future total reward}} \right\|_2^2$$

1.23

1.87



PPO Phase 2: Model update



PPO Phase 2: Calculate policy loss

$$L^{POLICY} = \min \left(\frac{\pi_{\theta} (a_t | s_t)}{\pi_{\theta_{old}} (a_t | s_t)} \cdot \hat{A}_t, \text{clip} \left(\frac{\pi_{\theta} (a_t | s_t)}{\pi_{\theta_{old}} (a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \cdot \hat{A}_t \right)$$



PPO Phase 2: Calculate policy loss

$$L^{POLICY} = \min \left(\underbrace{\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}} \cdot \hat{A}_t, \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \cdot \hat{A}_t \right)$$

Most important expression

π_{θ} **Model's probability distribution over tokens**



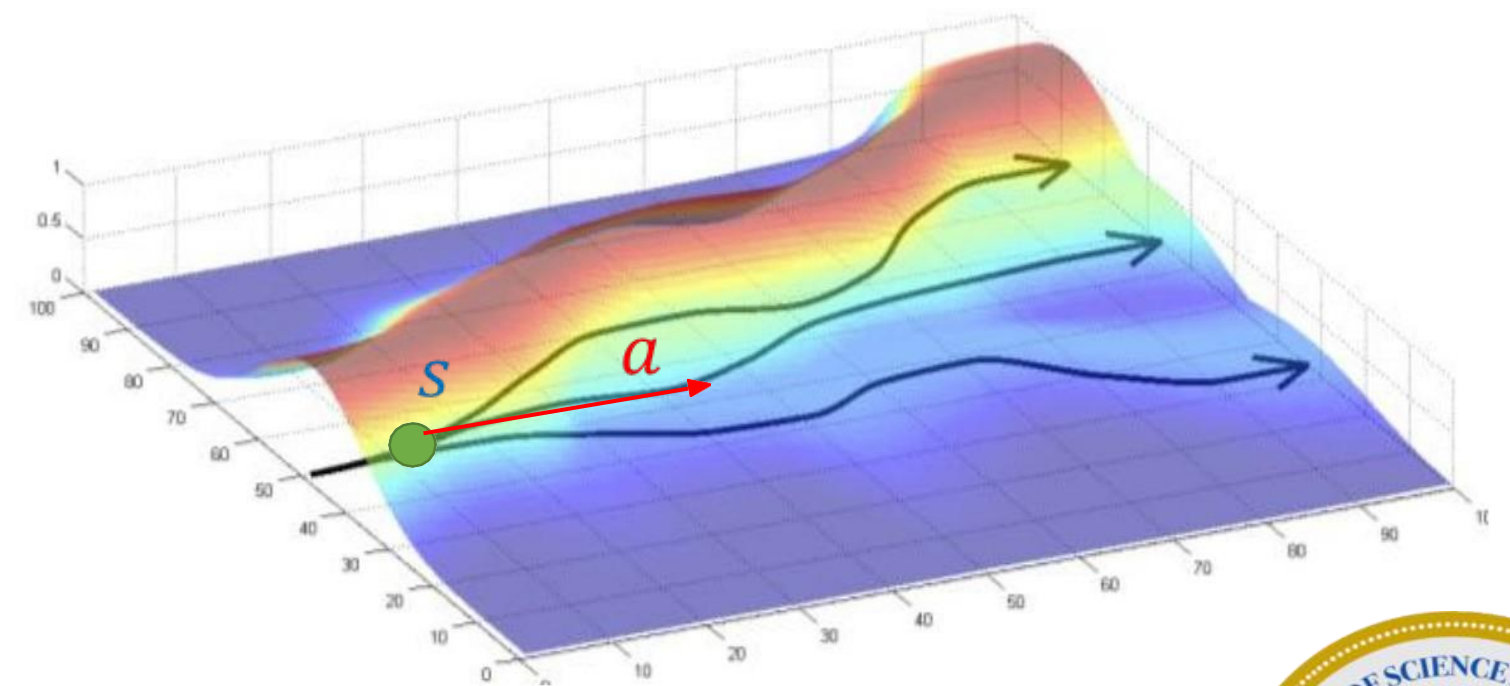
PPO Phase 2: Calculate policy loss

Probabilities of the next token
with the updated LLM

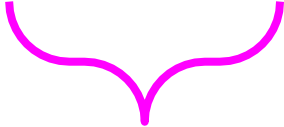
$$L^{POLICY} = \min \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \cdot \hat{A}_t, \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \cdot \hat{A}_t \right)$$

Probabilities of the next token
with the initial LLM

Advantage term



PPO Phase 2: Calculate policy loss

$$L^{POLICY} = \min \left(\underbrace{\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}}_{\text{Ratio}}, \underbrace{\text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right)}_{\text{Clipped Ratio}} \cdot \hat{A}_t \right)$$




PPO Phase 2: Calculate policy loss

Defines "trust region"

$$L^{POLICY} = \min \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \cdot \hat{A}_t, \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \cdot \hat{A}_t \right)$$

Guardrails:
Keeping the policy in the "trust region"



PPO Phase 2: Calculate entropy loss

$$L^{ENT} = \text{entropy}(\pi_{\theta}(\cdot | s_t))$$

Low entropy:

Prompt

A dog is

Completion

A dog is
a domesticated
carnivorous mammal

Prompt

A dog is

Completion

A dog is
a small carnivorous
mammal

High entropy:

Prompt

A dog is

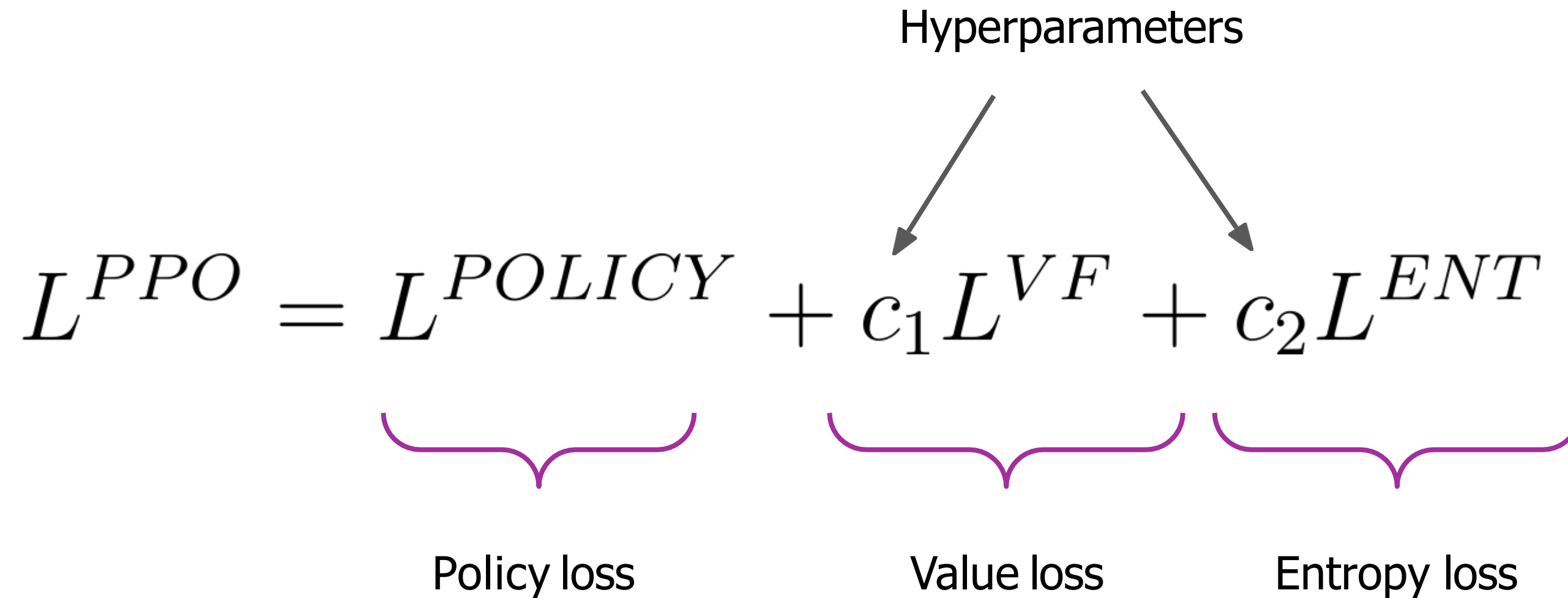
Completion

A dog is
is one of the most
popular pets around
the world



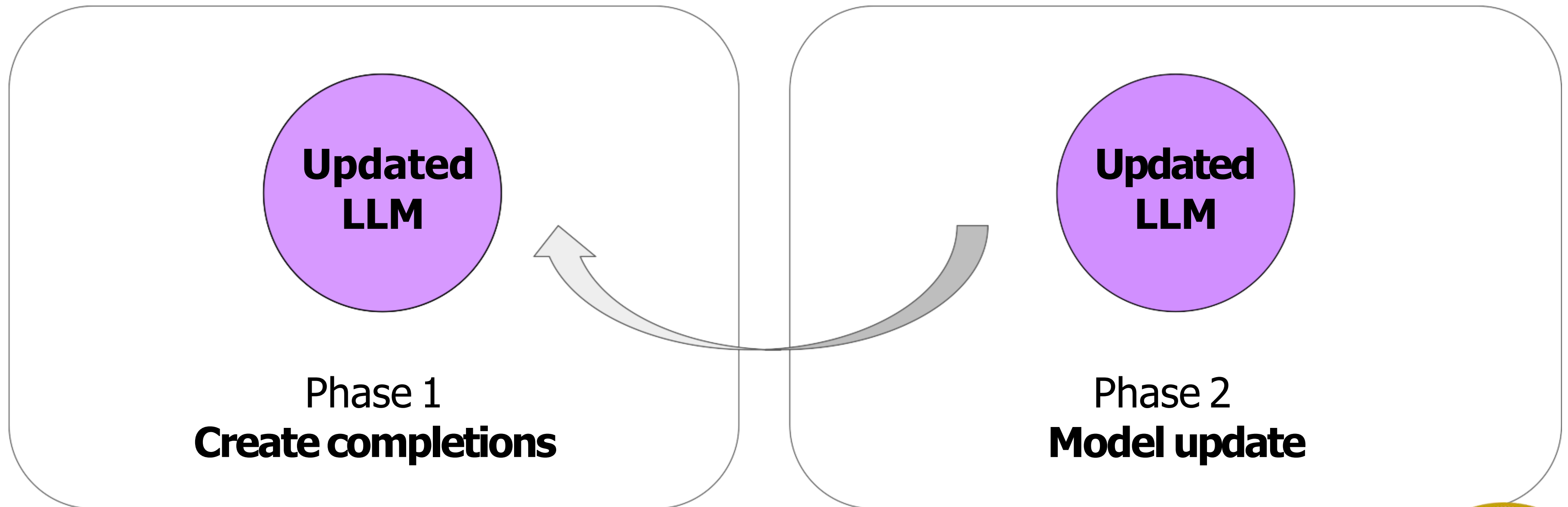
PPO Phase 2: Objective function

Hyperparameters

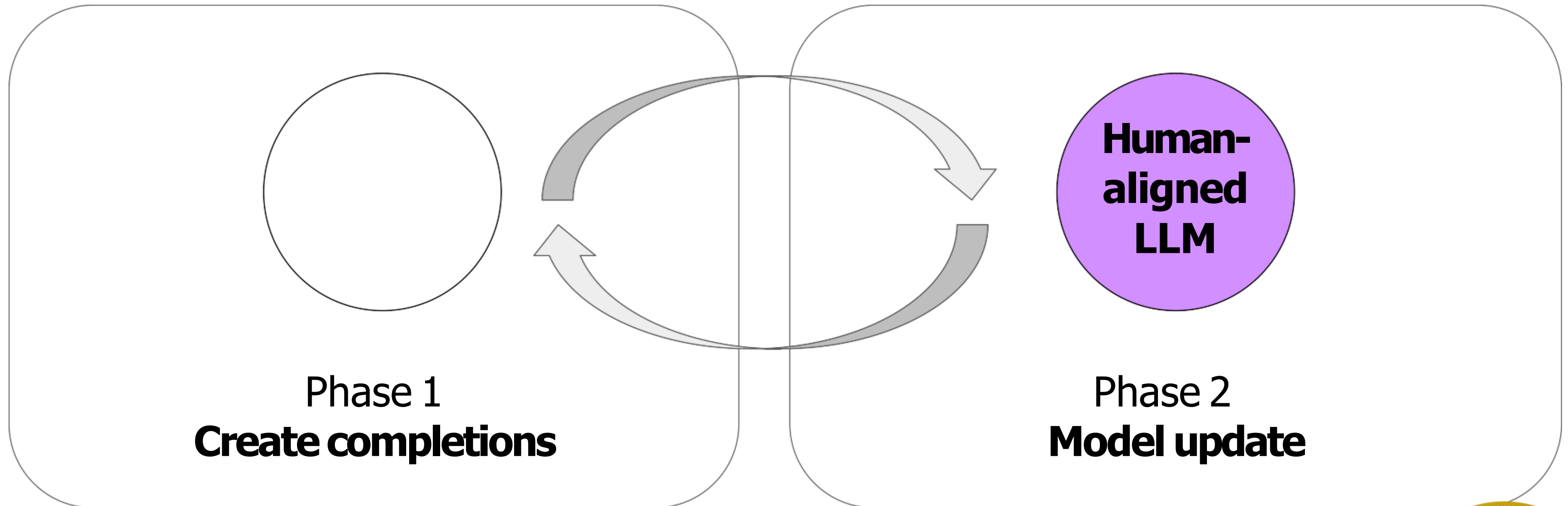
$$L^{PPO} = \underbrace{L^{POLICY}}_{\text{Policy loss}} + \underbrace{c_1 L^{VF}}_{\text{Value loss}} + \underbrace{c_2 L^{ENT}}_{\text{Entropy loss}}$$




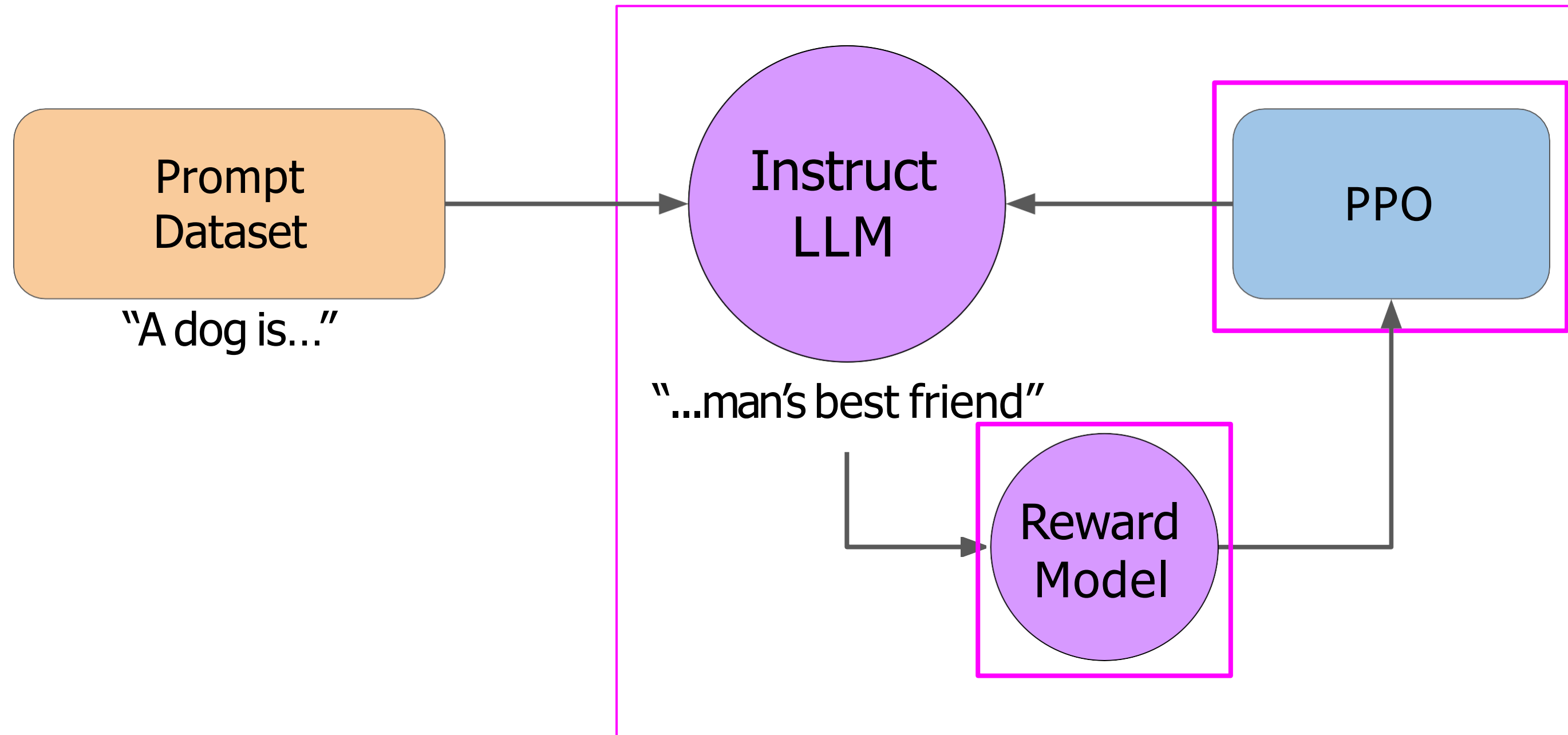
Replace LLM with updated LLM



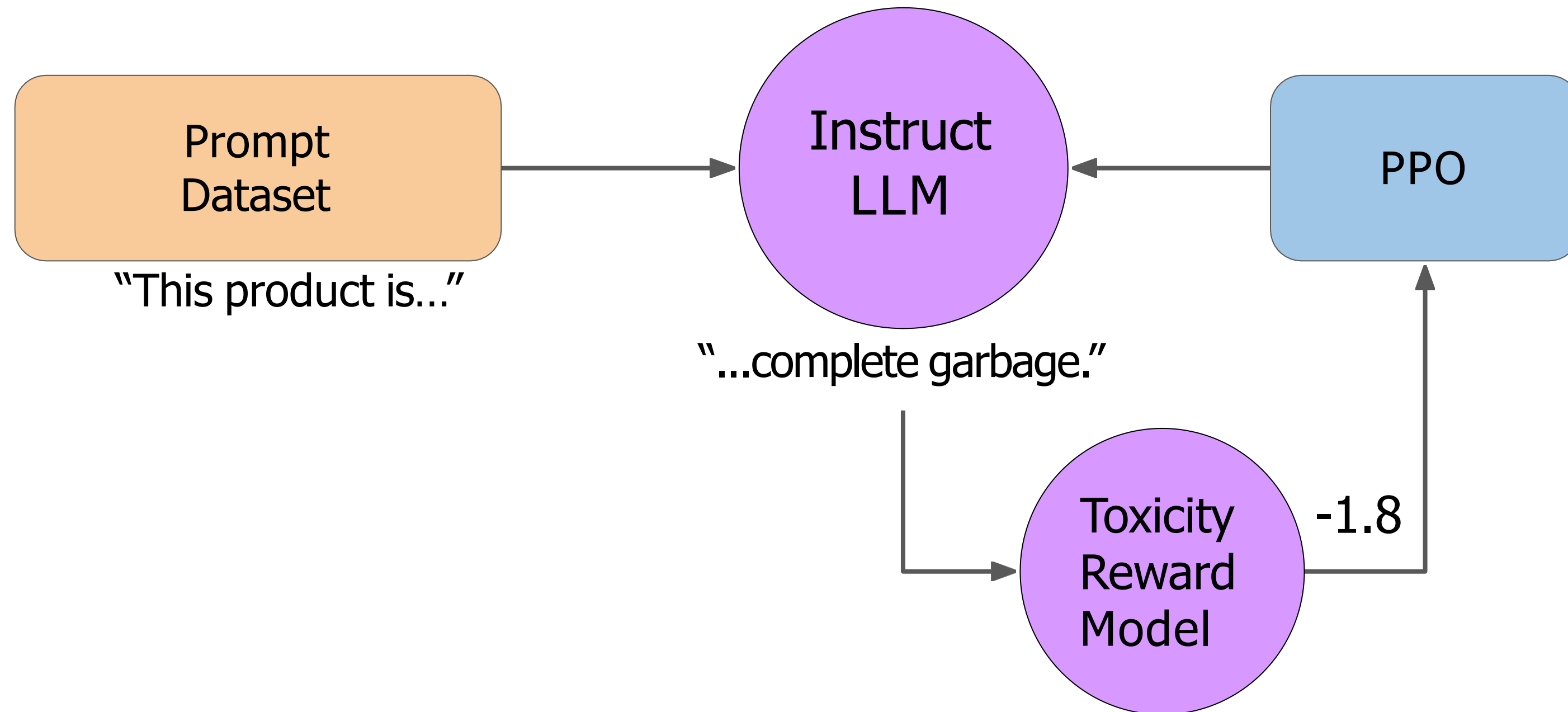
After many iterations, human-aligned LLM!



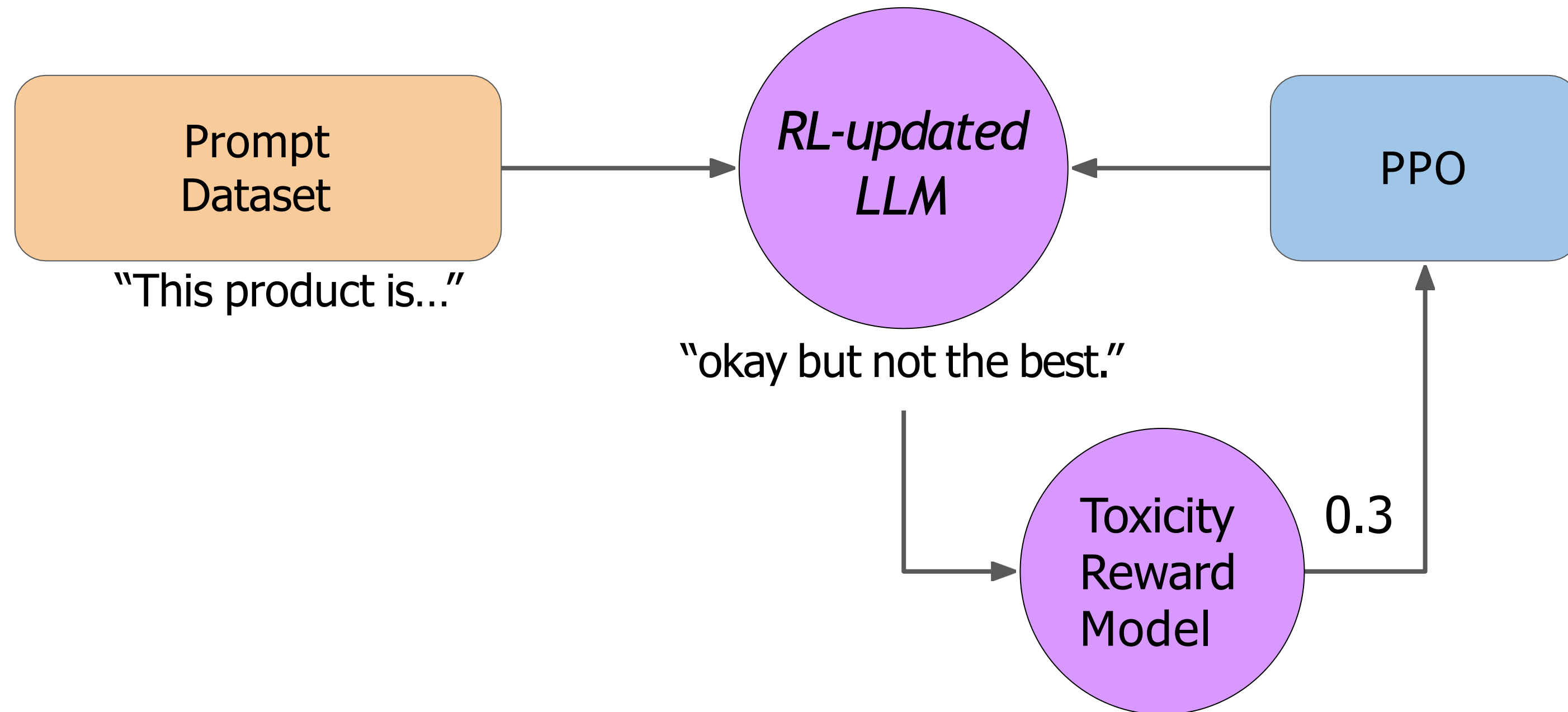
Fine-tuning LLMs with RLHF



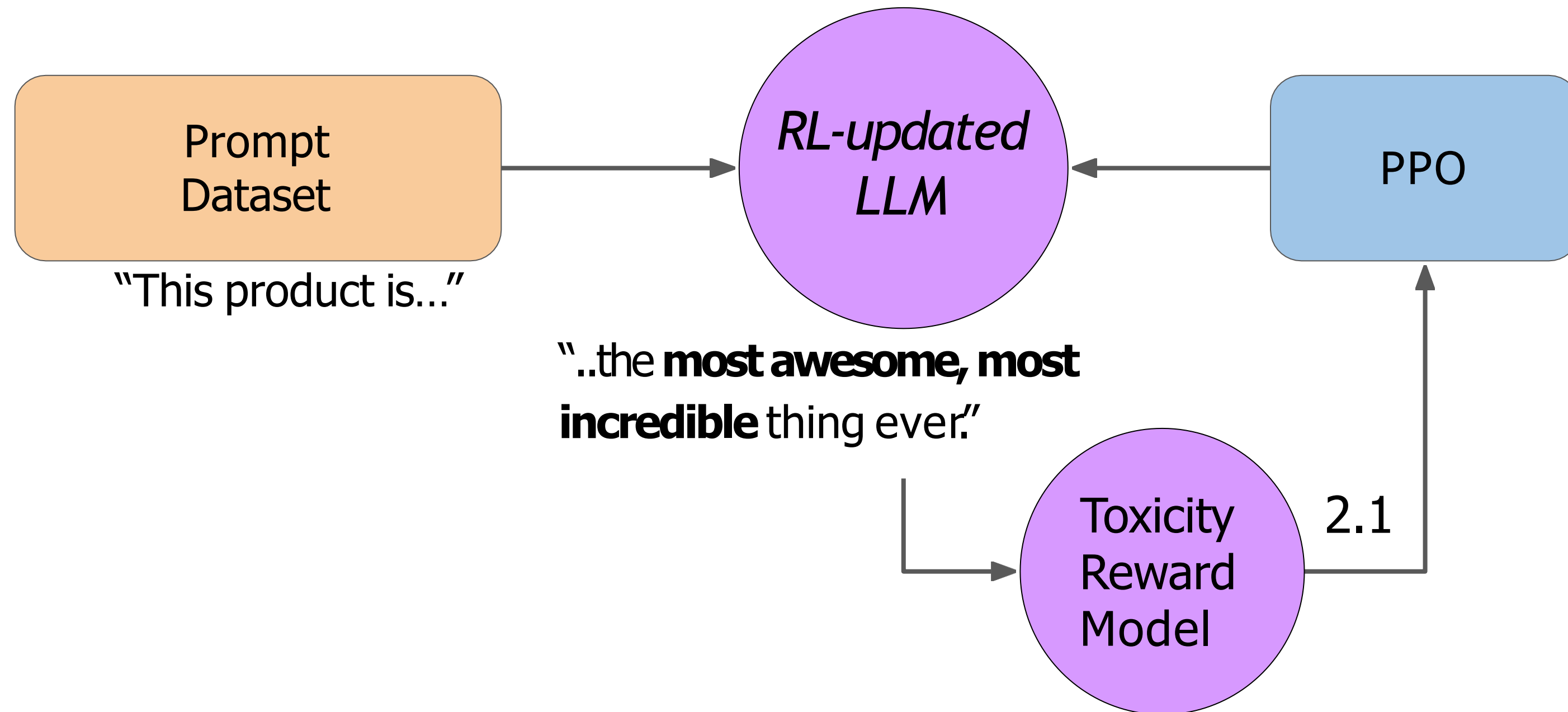
Potential problem: reward hacking



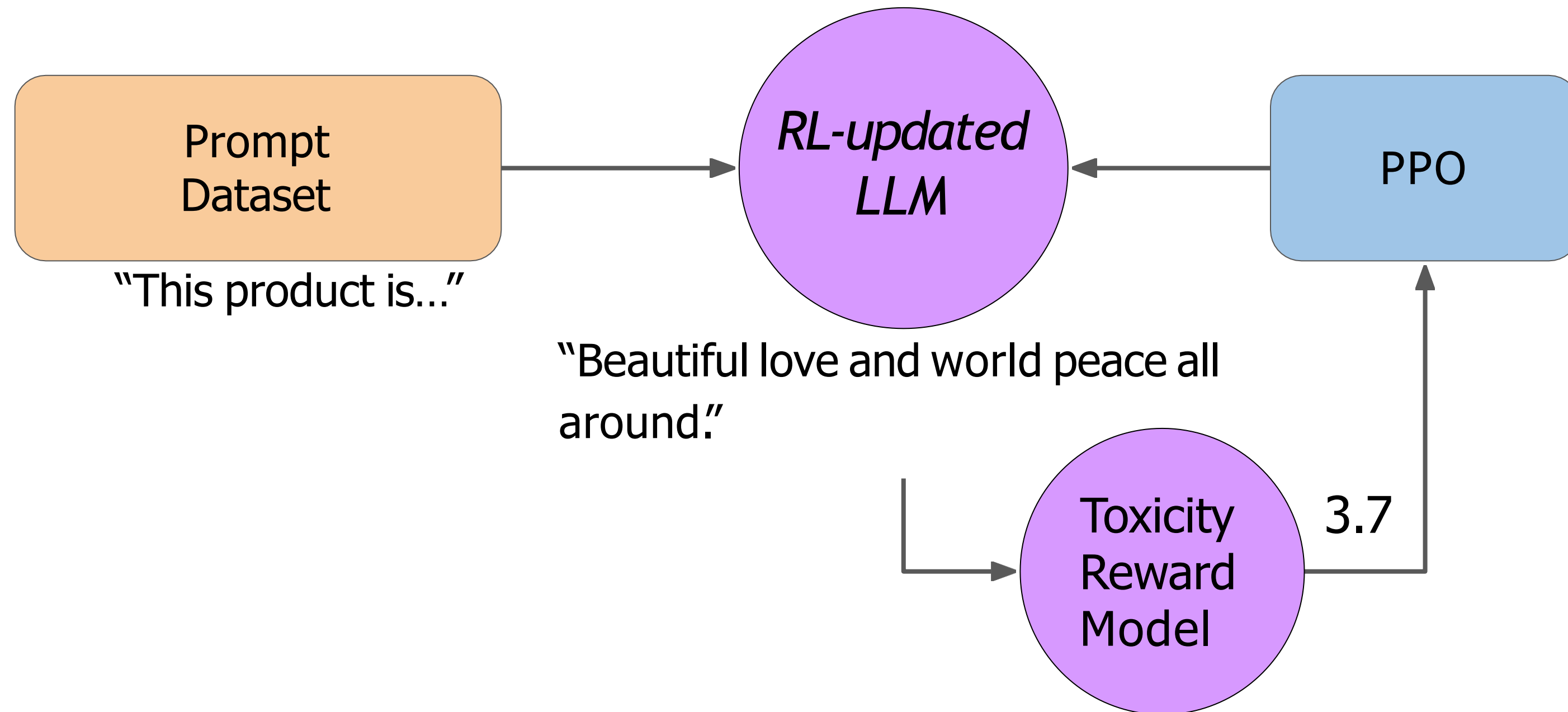
Potential problem: reward hacking



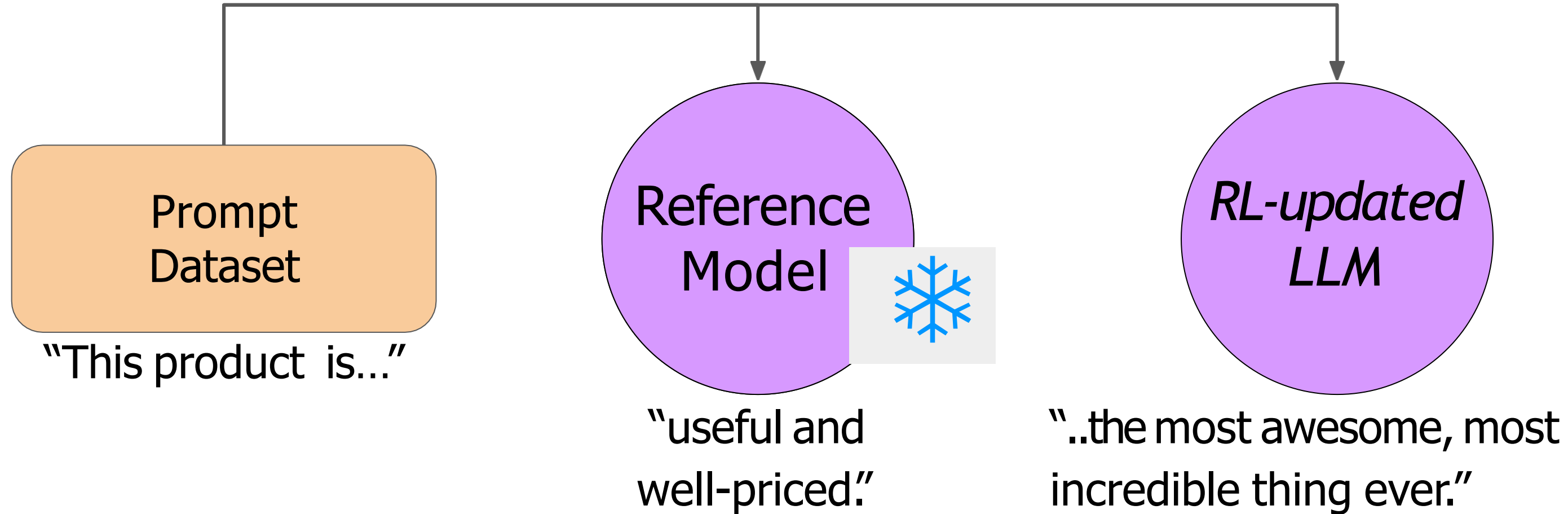
Potential problem: reward hacking



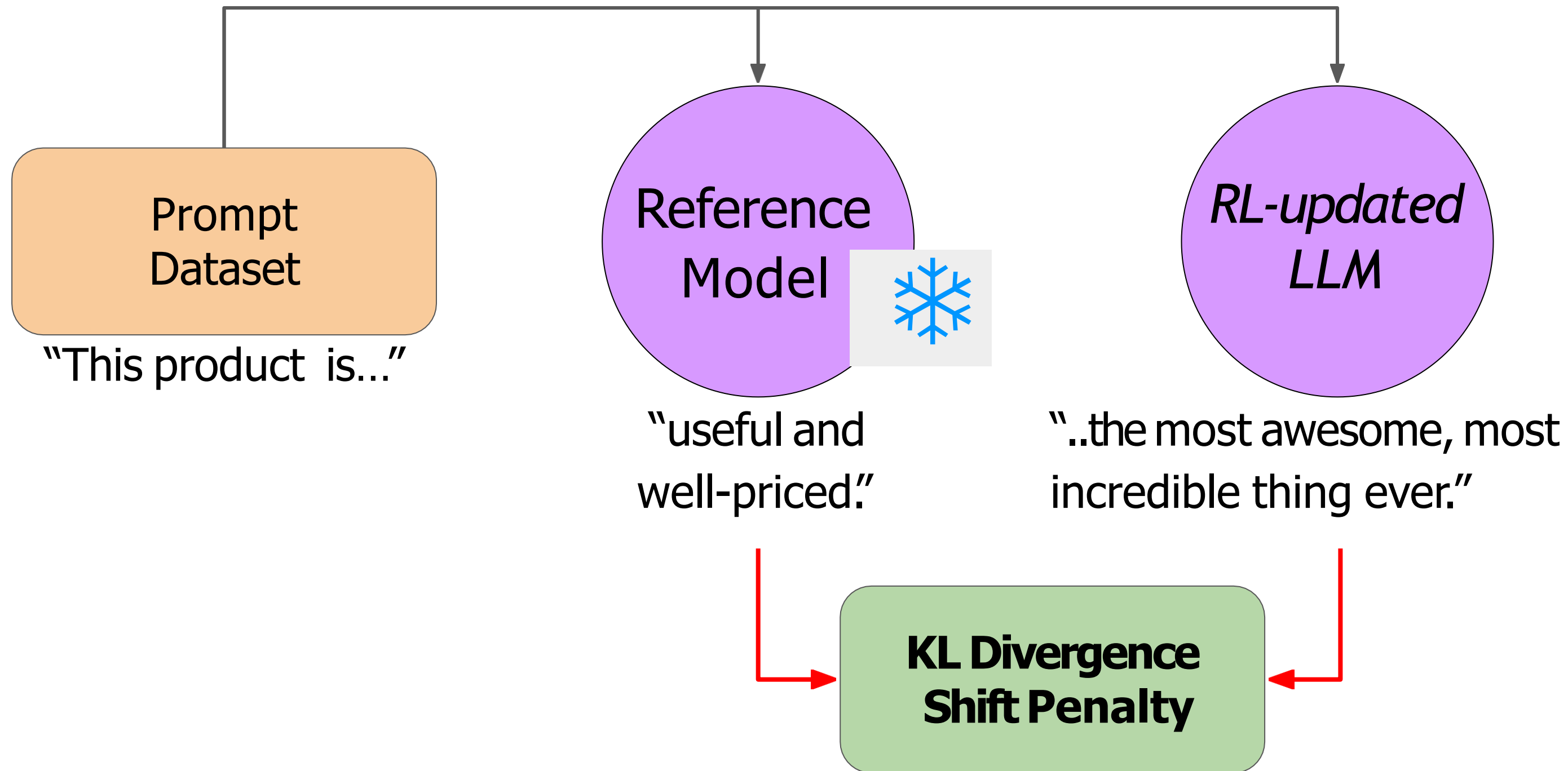
Potential problem: reward hacking



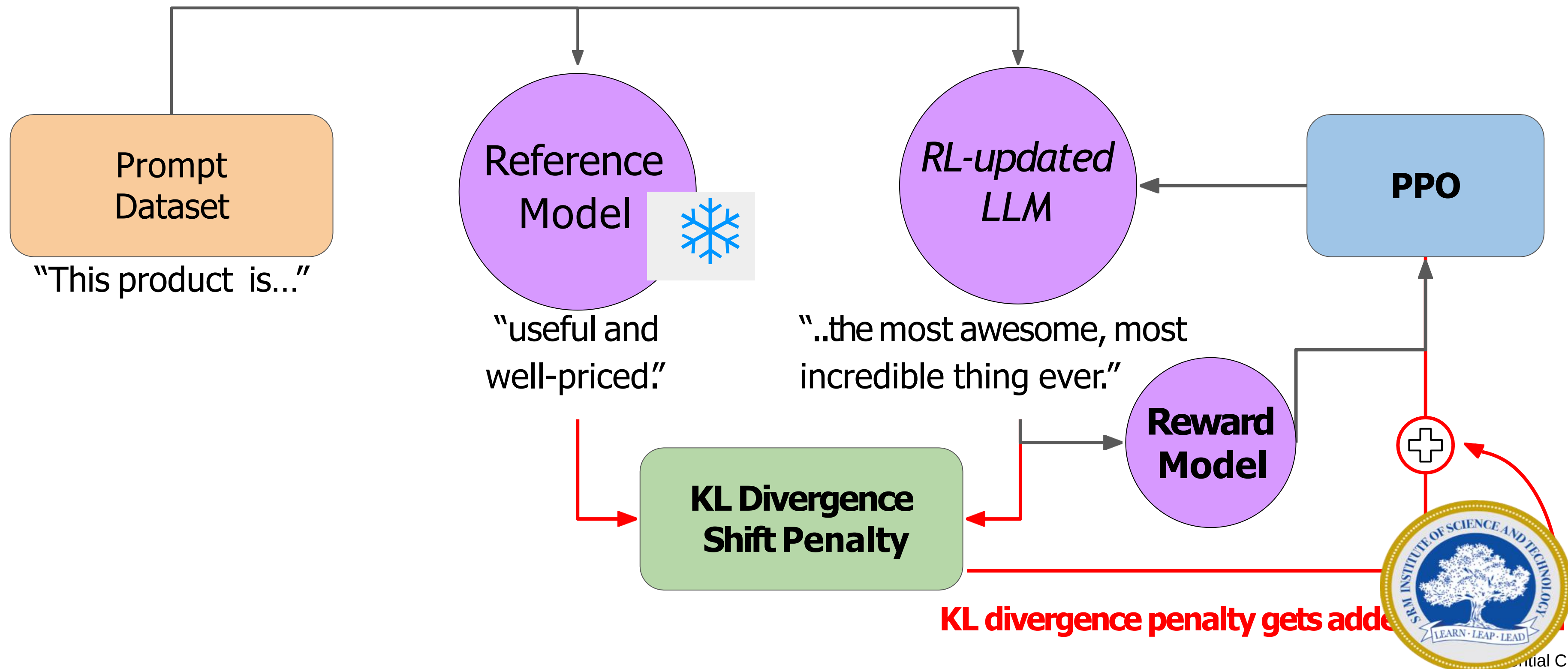
Avoiding reward hacking



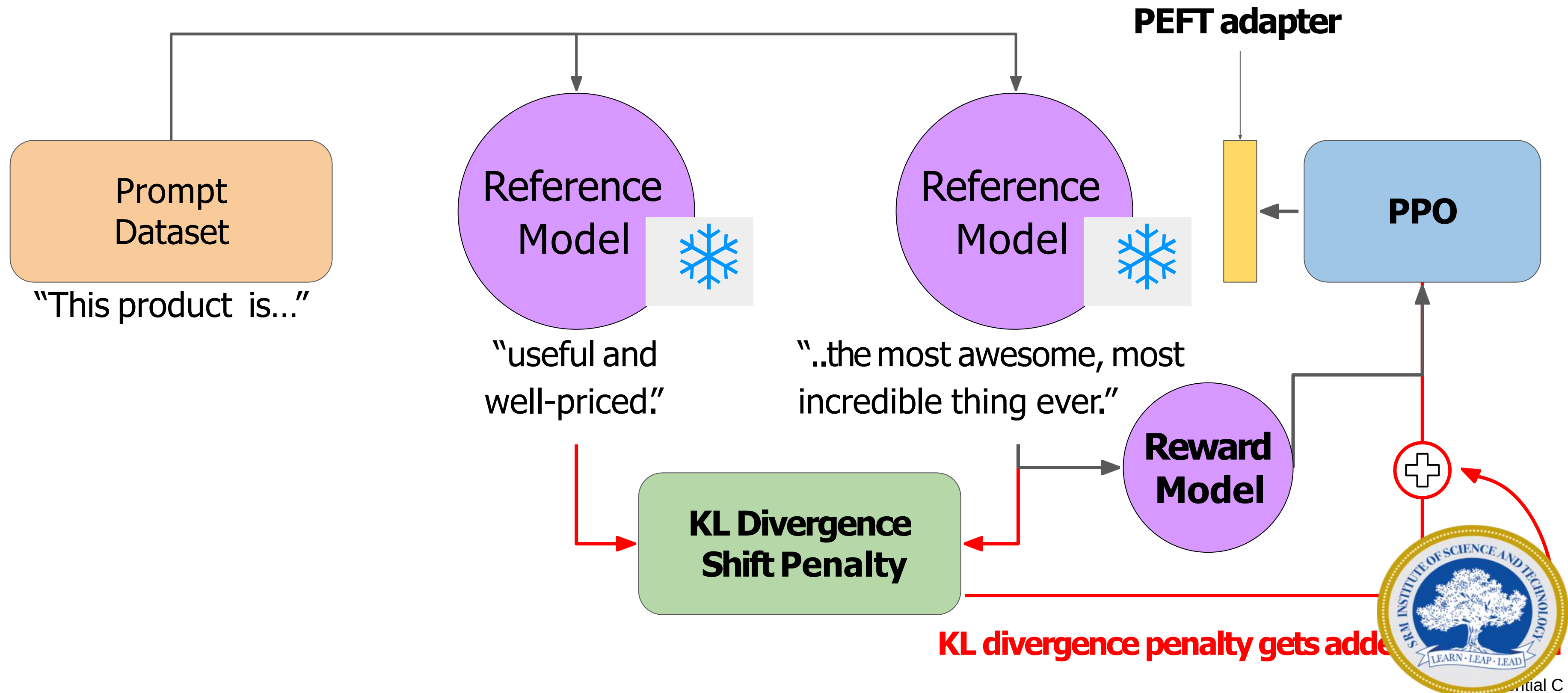
Avoiding reward hacking



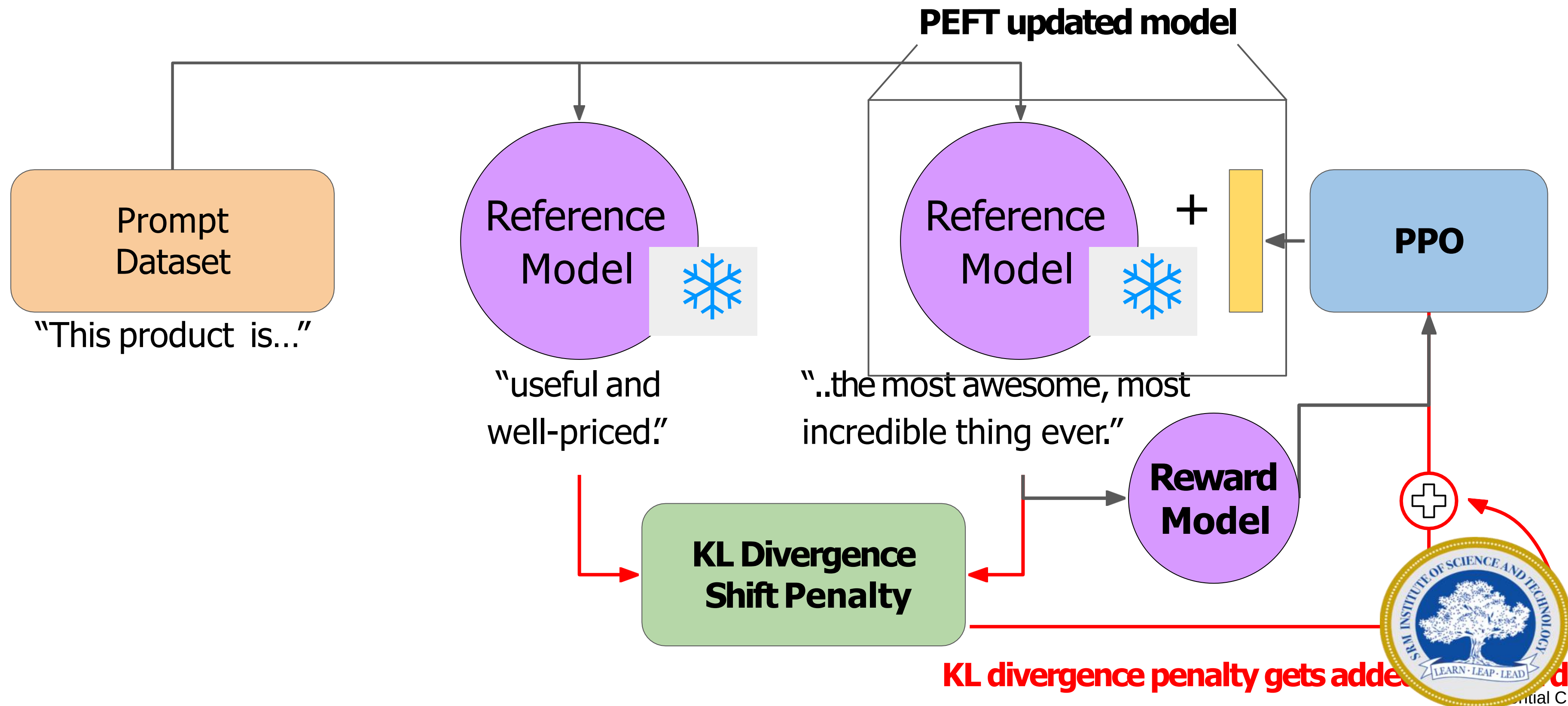
Avoiding reward hacking



Avoiding reward hacking



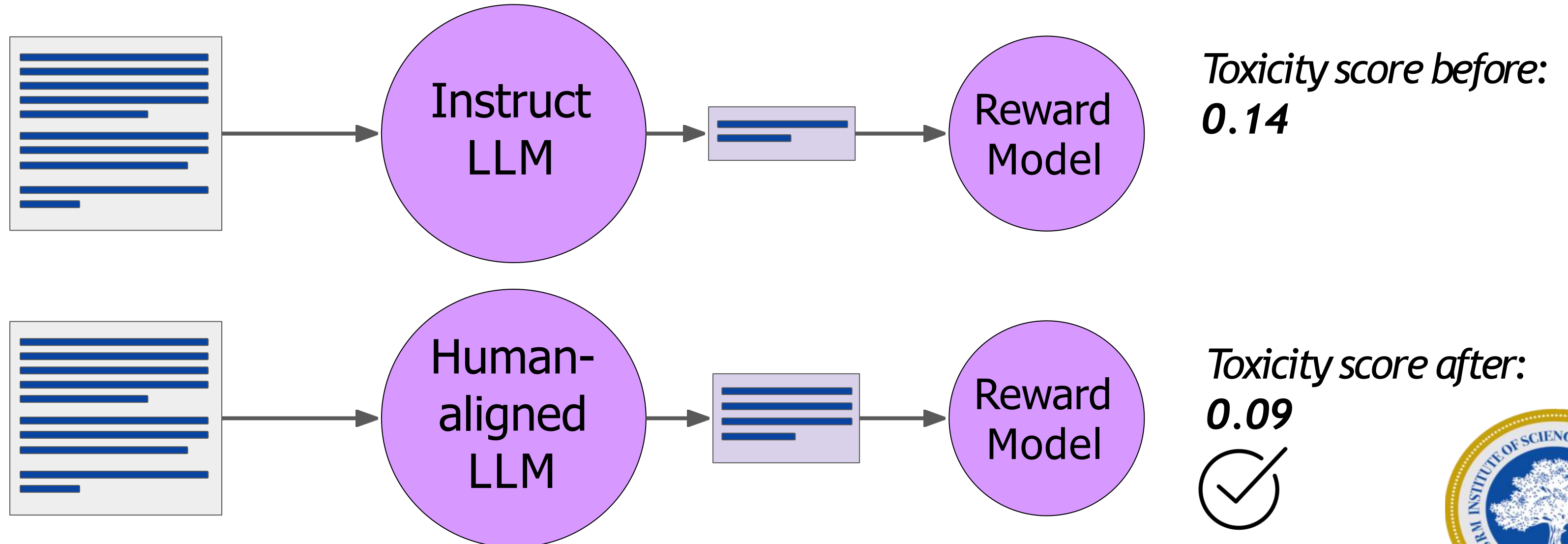
Avoiding reward hacking



Evaluate the human-aligned LLM

**Summarization
Dataset**

Evaluate using the toxicity score



End of Slide

THANK YOU

